Università Politecnica delle Marche

Programmazione Mobile
Corso di Laurea Triennale in Ingegneria Informatica e dell'Automazione

Reverie

Relatore:

Prof. Storti Emanuele

Tesina di:

Pelusi Alessio
Vagnarelli Beniamino

ANNO ACCADEMICO 2024/2025

Contents

1 Descrizione Progetto **1**

2 Glossario dei Termini **2**

3 Gestione dei Requisiti **3**
3.1 Requisiti Funzionali . . . 3
3.2 Requisiti Non Funzionali . . . 4
3.3 Diagrammi dei casi d'uso . . . 6

4 Progettazione **16**
4.1 Descrizione dell'architettura MVVM . . . 16
4.2 Descrizione dell'architettura DAO . . . 17
4.3 Firebase come Remote Data Source . . . 18
4.4 Diagramma delle Componenti . . . 19
4.5 Mockup . . . 20

5 Implementazione in Kotlin **23**
5.1 Struttura del File System del Progetto . . . 23
5.2 Approfondimenti sul codice . . . 24
5.2.1 Model . . . 24
5.2.2 Repository . . . 26
5.2.3 Navigation . . . 26
5.2.4 Screen . . . 26
5.2.5 Components . . . 28
5.2.6 Theme . . . 29
5.2.7 ViewModel . . . 29
5.3 Descrizione unit tests . . . 31
5.3.1 Mock repository . . . 31
5.3.2 Mock ServiceModule . . . 32
5.3.3 Classe AllDiariesScreenTest . . . 33
5.3.4 Classe AllTimeCapsulesScreenTest . . . 37

6 Implementazione in Flutter **41**
6.1 Struttura del File System del Progetto . . . 41
6.2 Approfondimenti sul codice . . . 42
6.3 Descrizione unit tests . . . 43
6.3.1 all_diaries_screen_test.dart . . . 43
6.3.2 all_time_capsule_screen_test.dart . . . 45
6.3.3 edit_diary_screen_test.dart . . . 47
6.3.4 create_time_capsule_screen_test.dart . . . 49

7 Conclusione **51**

1. Descrizione Progetto

L'app ha l'obiettivo di offrire agli utenti uno spazio personale in cui conservare ricordi, pensieri ed emozioni attraverso diari digitali. Ogni diario è identificato da un titolo, una descrizione e una copertina. Per ciascun giorno, l'utente può scrivere un racconto quotidiano con la possibilità di inserire anche delle immagini.

Una funzionalità distintiva dell'app è la creazione delle "capsule del tempo": messaggi programmati da inviare nel futuro. Queste lettere, una volta scritte, restano nascoste fino alla data prestabilita, per poi essere recapitate via email o SMS. L'intento è quello di rivivere un momento passato o condividere un pensiero con se stessi o con altre persone.
L'utente può scegliere la data di consegna e i destinatari, specificandoli tramite indirizzo email, numero di telefono o username collegato ad un altro account della piattaforma. Alla scadenza, il messaggio verrà automaticamente inviato o sarà reso direttamente disponibile nell'app.

2. Glossario dei Termini

Termine	Descrizione	Tipo	Sinonimi
Diario	Forma narrativa in cui il racconto, sviluppato cronologicamente ad intervalli giornalieri, contiene avvenimenti personali e importanti.	Business	Nessuno
Racconto quotidiano	Insieme di pagine che vanno a costituire un giorno nel diario.	Business	Cronaca quotidiana, Pensiero quotidiano, Ricordo del giorno, Riflessione giornaliera
Pagina di diario	Singola pagina del diario.	Business	Pagina
Capsula del tempo	Messaggio o pensiero da far rivivere nel futuro	Business	Lettera
Indirizzo Email	Identificatore unico utilizzato per inviare e ricevere messaggi elettronici tramite Internet.	Tecnico	Indirizzo di posta elettronica, email
Login	Processo con cui un utente viene identificato e entra nella piattaforma.	Tecnico	Accesso
Sign up	Processo di creazione di un profilo utente utilizzato dal cliente per effettuare ordini e inserire recensioni.	Tecnico	Registrazione account, creazione account
Account	Area personale dell'utente	Tecnico	Profilo

3. Gestione dei Requisiti

3.1 Requisiti Funzionali

Area: Gestione Utenti
RF1: Signup Utente
Il sistema dovrà permettere all'utente di creare il proprio account.

RF2: Login Utente
Il sistema dovrà permettere all'utente di accedere al proprio account.

RF3: Reset password Account
Il sistema dovrà permettere all'utente di eseguire il reset della password dell'account.

RF4: Modifica Account Utente
Il sistema dovrà permettere all'utente di modificare le informazioni del proprio account.

Area: Gestione Diari
RF5: Visualizza lista diari
Il sistema dovrà permettere la visualizzazione di tutti i diari dell'utente.

RF6: Crea nuovo diario
Il sistema dovrà permette all'utente di creare un nuovo diario.

RF7: Elimina diario
Il sistema dovrà permette all'utente di eliminare un diario.

RF8: Visualizza diario
Il sistema dovrà permettere all'utente la visualizzazione del diario selezionato.

RF9: Modifica diario
Il sistema dovrà permettere all'utente di modificare titolo e descrizione del diario selezionato e di cambiare la sua copertina.

RF10: Aggiungi racconto quotidiano
Il sistema dovrà permettere all'utente di aggiungere nel diario selezionato un nuovo racconto quotidiano relativo al nuovo giorno.

RF11: Modifica racconto quotidiano
Il sistema dovrà permettere all'utente di modificare il testo del racconto quotidiano.

RF12: Inserisci immagini al racconto quotidiano
Il sistema dovrà permettere all'utente di inserire immagini nel racconto quotidiano.

RF13: Elimina racconto quotidiano
Il sistema dovrà permettere all'utente di eliminare il proprio racconto quotidiano.

Area: Gestione Capsule del tempo
RF14: Creazione nuova capsula del tempo
Il sistema dovrà permettere all'utente di creare una nuova capsula del tempo.

RF15: Selezione destinatari capsula del tempo
Il sistema dovrà permettere all'utente di selezionare molteplici destinatari della nuova capsula del tempo.

RF16: Selezione data di consegna capsula del tempo
Il sistema dovrà permettere all'utente di selezionare la data di consegna della nuova capsula del tempo.

RF17: Visualizzazione capsula del tempo
Il sistema dovrà permettere all'utente di visualizzare la capsula del tempo, il cui contenuto sarà accessibile solo a partire dalla data di consegna.

RF18: Visualizzazione capsule del tempo programmate
Il sistema dovrà permettere all'utente di visualizzare le capsule del tempo da lui create che aspettano di essere inviate ai riceventi dopo la data di scadenza.

RF18: Visualizzazione capsule del tempo inviate
Il sistema dovrà permettere all'utente di visualizzare le capsule del tempo da lui create e inviate.

RF19: Visualizzazione capsule del tempo ricevute
Il sistema dovrà permettere all'utente di visualizzare le capsule del tempo da lui ricevute.

RF20: Eliminazione capsula del tempo
Il sistema dovrà permettere all'utente di eliminare le capsule del tempo non ancora consegnate.

3.2 Requisiti Non Funzionali

Area: Gestione Tecnologie - vincoli
RNF1: Utilizzo di un Database non relazionale
Il sistema dovrà adoperare un database non relazionale per il salvataggio dei dati.

RNF2: Supporto minimo per Android 13
Il sistema dovrà supportare minimo la versione 13 di Android.

RNF3: Supporto minimo API 33
Il sistema dovrà supportare minimo la versione delle API 33 di Android.

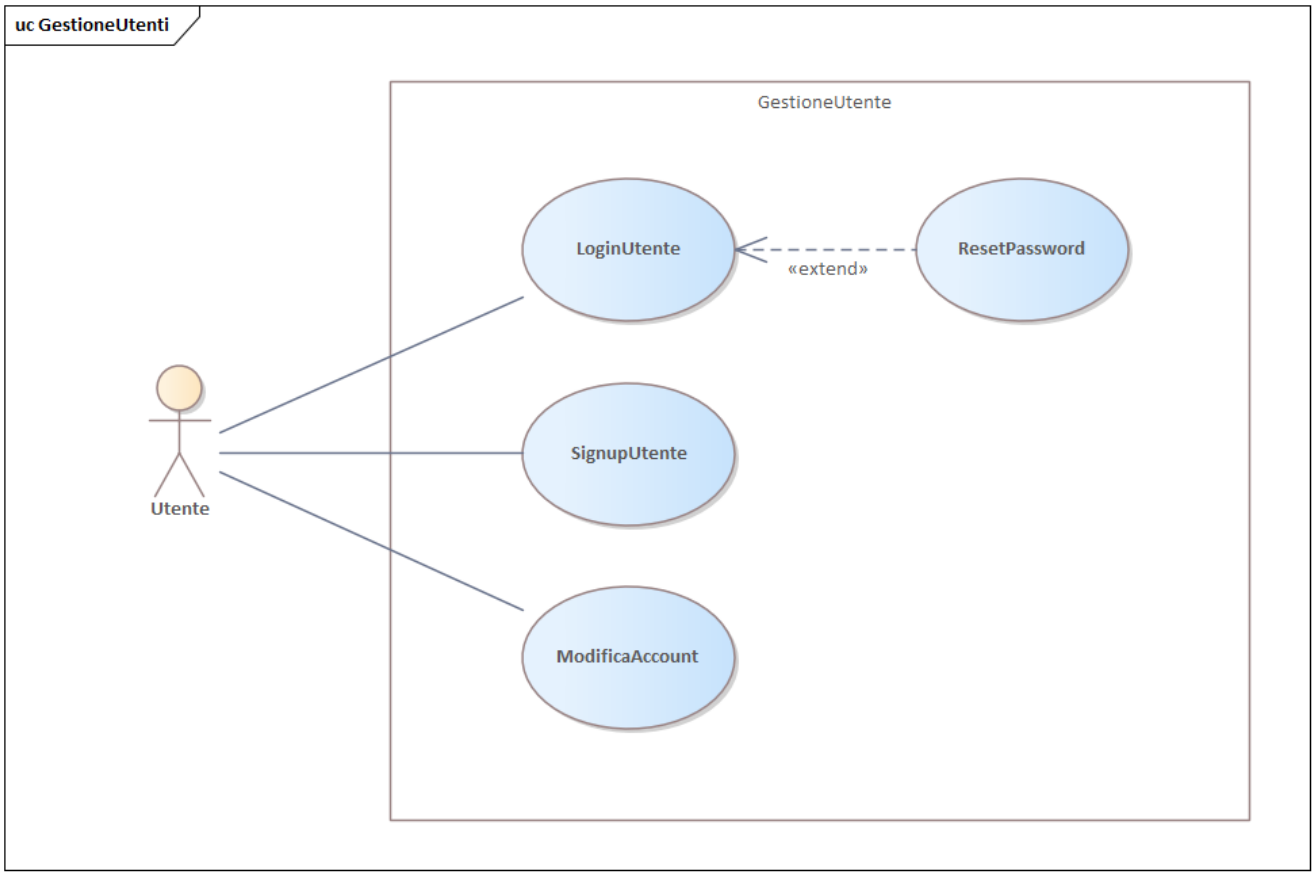
RNF4: Utilizzo framework Jetpack Compose
Il progetto utilizzerà il framework Jetpack Compose per l'implementazione nativa in Android.

RNF5: Utilizzo framework Flutter
Il progetto utilizzerà il framework Flutter per l'implementazione cross platform.

RNF6: Modalità di ricezione capsule del tempo
Il sistema dovrà supportare l'invio delle capsule del tempo via email e sms.

3.3 Diagrammi dei casi d'uso

Diagramma Gestione Utente



Caso d'uso: SignupUtente

Caso d'uso: SignupUtente
ID: 1
Breve Descrizione: Il sistema crea un nuovo account per l'utente.
Attori Primari: Utente
Attori Secondari: Nessuno
Precondizioni: Nessuna
Sequenza degli eventi principale: 1. Il caso d'uso inizia quando l'Utente seleziona "Crea nuovo account". 2. **While** le informazioni dell'Utente non sono valide 2.1 Il sistema chiede all'Utente di inserire le sue informazioni: username, nome, cognome, email, password e password di conferma. 2.2 Il sistema valida le informazioni dell'Utente. 3. Il sistema crea un nuovo account per l'Utente.
Postcondizioni: 1. Un nuovo account è stato creato per l'Utente.
Sequenza degli eventi alternativa: Nessuna

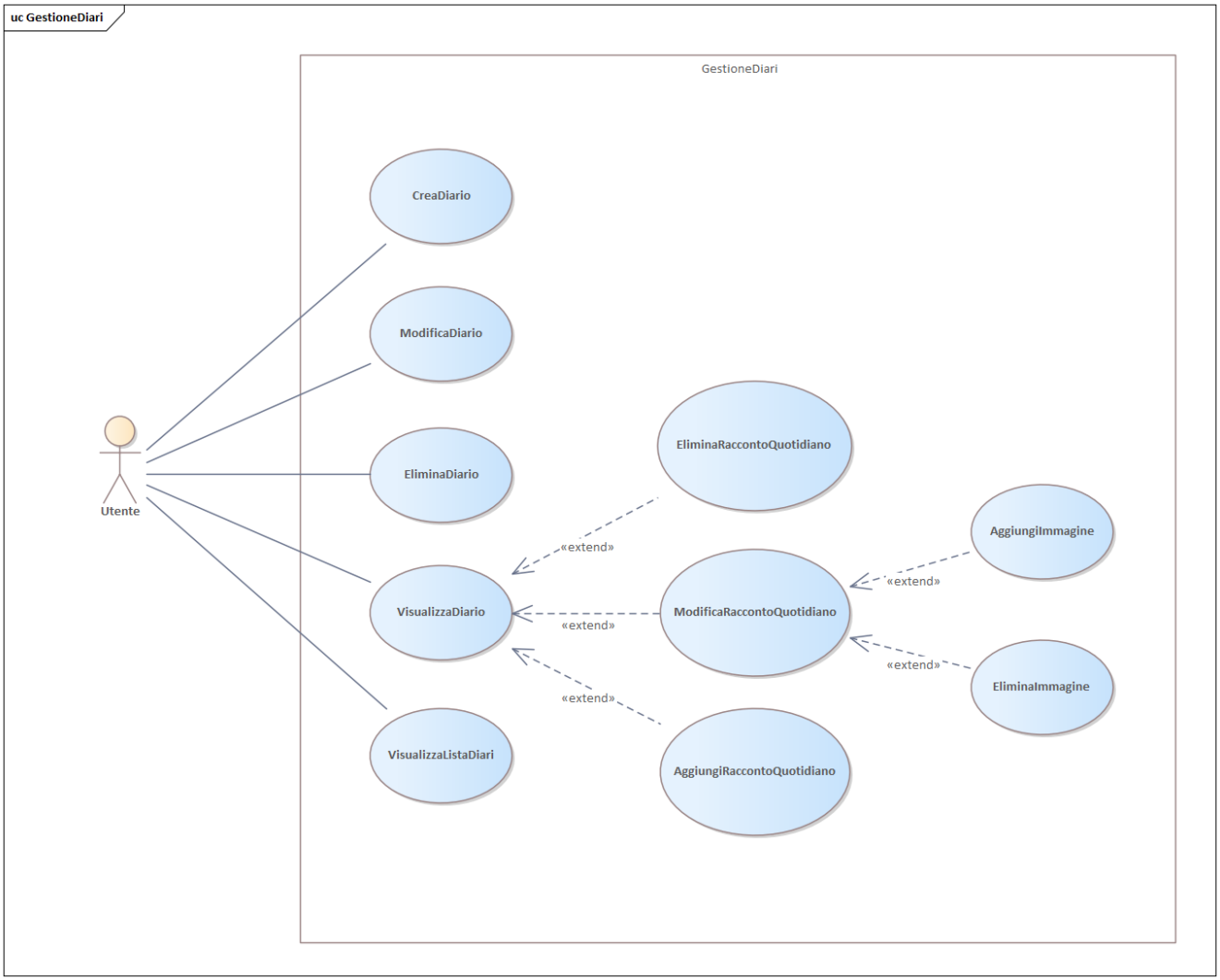
Caso d'uso: LoginUtente

Caso d'uso: LoginUtente
ID: 2
Breve Descrizione: Il sistema permette all'Utente di accedere al proprio account.
Attori Primari: Utente
Attori Secondari: Nessuno
Precondizioni: 1. L'Utente deve essere già registrato.
Sequenza degli eventi principale: 1. Il caso d'uso inizia quando l'Utente seleziona "Accedi". punto di estensione: Reset Password 2. **While** le credenziali inserite dall'Utente non sono valide 2.1 Il sistema chiede all'Utente di inserire email e password. 2.2 Il sistema valida le credenziali inserite dall'Utente. 3. Il sistema autentica l'Utente.
Postcondizioni: 1. L'Utente ha effettuato il login al proprio account.
Sequenza degli eventi alternativa: Nessuna

Caso d'uso: ModificaAccount

Caso d'uso: ModificaAccount
ID: 3
Breve Descrizione: Il sistema permette all'Utente di modificare le informazioni del proprio account.
Attori Primari: Utente
Attori Secondari: Nessuno
Precondizioni: 1. L'Utente deve essere registrato alla piattaforma. 2. L'Utente deve aver effettuato l'accesso al proprio account. 3. L'Utente deve visualizzare la pagina del profilo.
Sequenza degli eventi principale: 1. Il caso d'uso inizia quando l'Utente seleziona "Modifica Account". 2. **While** le informazioni inserite dall'Utente non sono valide: 2.1 L'Utente modifica le informazioni del proprio account. 2.2 Il sistema valida le informazioni inserite dall'Utente. 3. Il sistema aggiorna le informazioni del profilo dell'Utente.
Postcondizioni: 1. Le informazioni dell'account dell'Utente sono state aggiornate.
Sequenza degli eventi alternativa: Nessuna

Diagramma Gestione Diari



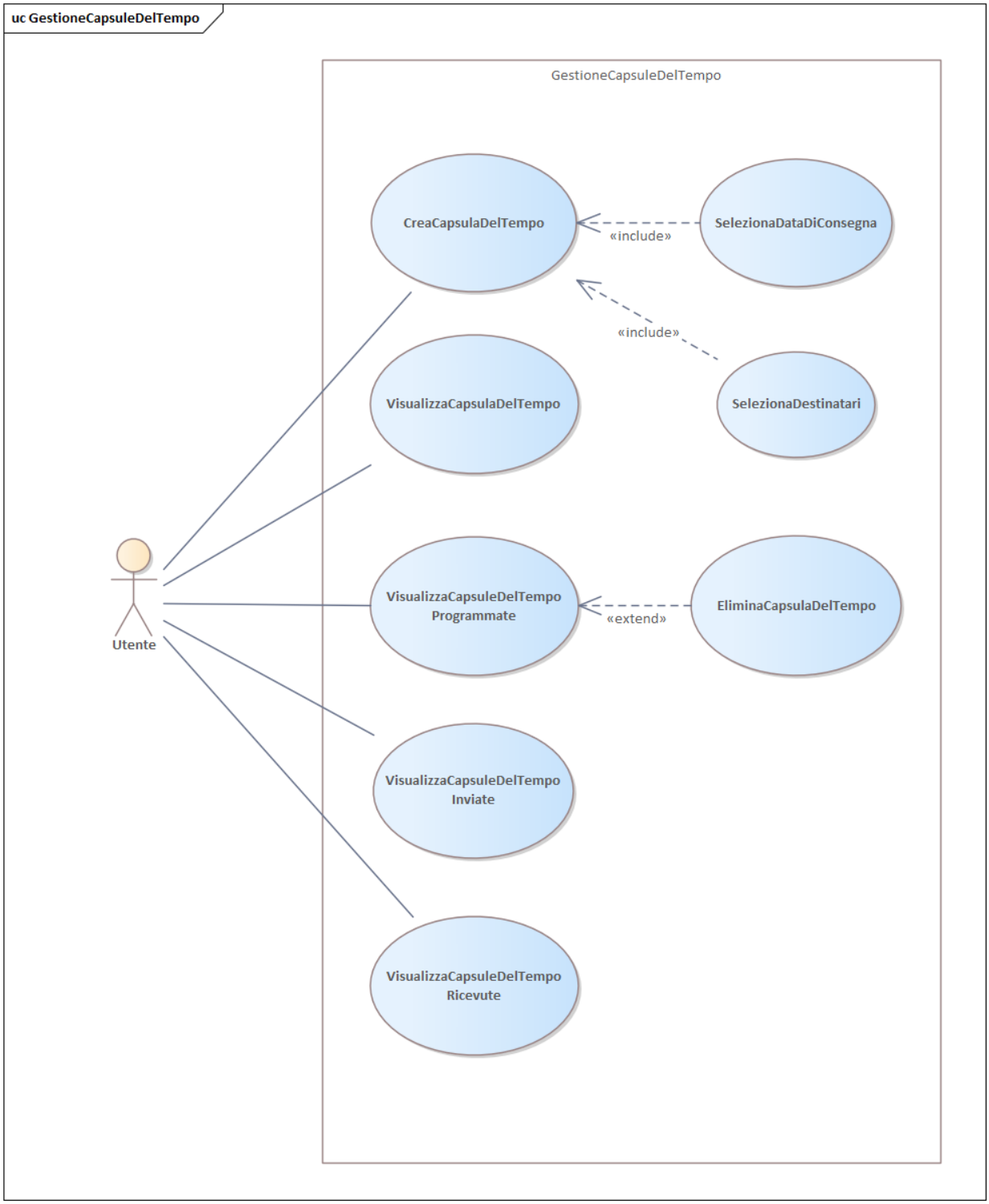
Caso d'uso: CreaDiario

Caso d'uso: CreaDiario
ID: 4
Breve Descrizione: L'Utente crea un nuovo diario.
Attori Primari: Utente
Attori Secondari: Nessuno
Precondizioni: 1. L'Utente ha effettuato il login al proprio account.
Sequenza degli eventi principale: 1. Il caso d'uso inizia quando l'Utente seleziona "Crea Diario". 2. Il sistema richiede all'Utente di inserire le seguenti informazioni del nuovo diario: titolo, descrizione e copertina. 3. L'utente inserisce le informazioni del diario richieste. 4. Il sistema crea il nuovo diario dell'Utente.
Postcondizioni: 1. Il nuovo diario è stato creato.
Sequenza degli eventi alternativa: Nessuna

Caso d'uso: ModificaRaccontoQuotidiano

Caso d'uso: ModificaRaccontoQuotidiano
ID: 5
Breve Descrizione: L'Utente modifica un racconto quotidiano del diario selezionato.
Attori Primari: Utente
Attori Secondari: Nessuno
Precondizioni: 1. L'Utente ha effettuato il login al proprio account. 2. L'Utente ha selezionato un racconto quotidiano di un proprio diario.
Sequenza degli eventi principale: 1. Il caso d'uso inizia quando l'Utente seleziona "Modifica Racconto Quotidiano". 2. L'Utente modifica il testo del racconto quotidiano selezionato. punto di estensione: Aggiungi Immagine punto di estensione: Elimina Immagine 3. Il sistema aggiorna il racconto quotidiano.
Postcondizioni: 1. Il racconto quotidiano è stato aggiornato.
Sequenza degli eventi alternativa: Nessuna

Diagramma Gestione Capsule del Tempo



Caso d'uso: CreaCapsulaDelTempo

Caso d'uso: CreaCapsulaDelTempo
ID: 6
Breve Descrizione: L'Utente crea una nuova capsula del tempo.
Attori Primari: Utente
Attori Secondari: Nessuno
Precondizioni: 1. L'Utente ha effettuato il login al proprio account.
Sequenza degli eventi principale: 1. Il caso d'uso inizia quando l'Utente seleziona "Crea Capsula del Tempo". 2. Il sistema richiede all'Utente di inserire le seguenti informazioni del nuovo prodotto: titolo e contenuto, di selezionare i destinatari e di selezionare la data di consegna della capsula. punto di inclusione: Seleziona Destinatari punto di inclusione: Seleziona Data di Consegna 3. L'utente inserisce le informazioni della capsula del tempo richieste. 4. Il sistema crea la nuova capsula del tempo dell'Utente.
Postcondizioni: 1. La capsula del tempo è stato creata.
Sequenza degli eventi alternativa: Nessuna

Caso d'uso di inclusione: SelezionaDestinatari

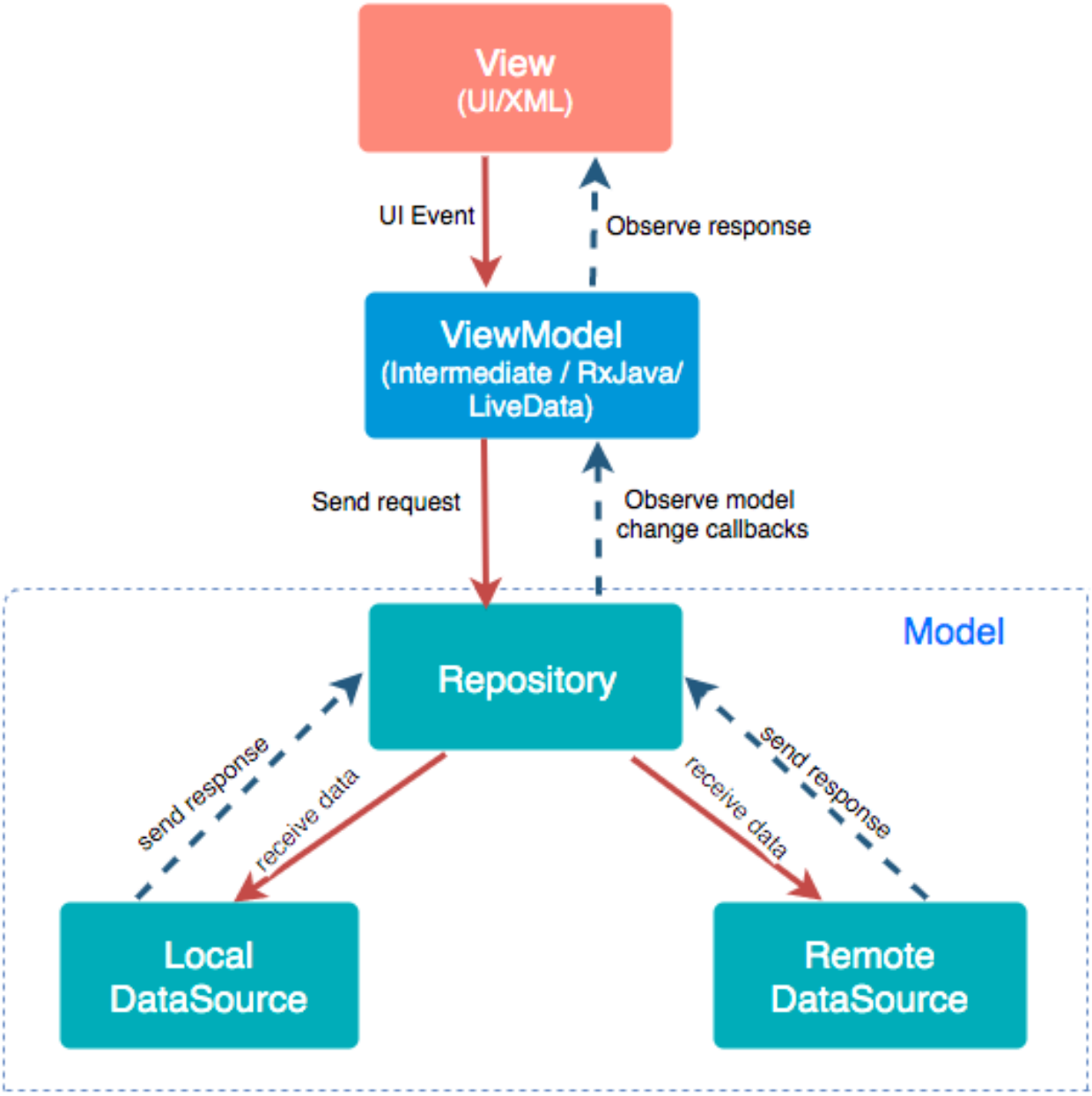
Caso d'uso di estensione: SelezionaDestinatari
ID: 7
Breve Descrizione: Segmento 1: L'Utente inserisce i destinatari della nuova capsula del tempo.
Attori Primari: Utente
Attori Secondari: Nessuno
Precondizioni del segmento 1: 1. L'Utente visualizza la schermata di creazione di una capsula del tempo.
Sequenza degli eventi del segmento 1: 1. L'Utente inserisce i destinatari della capsula del tempo, tramite una delle seguenti modalità: indirizzo email, numero di telefono, username dell'account. 2. Il sistema mostra all'Utente i destinatari inseriti.
Postcondizioni del segmento 1: 1. I destinatari della nuova capsula del tempo sono stati definiti.

4. Progettazione

4.1 Descrizione dell'architettura MVVM

L'architettura dell'applicazione si basa sull'utilizzo del pattern MVVM (Model–View–ViewModel), un modello architetturale ampiamente adottato nello sviluppo di applicazioni mobile moderne, sia in Flutter che su Android nativo. Questo pattern consente di strutturare l'app in modo modulare, separando in modo netto l'interfaccia utente dalla logica di business e dalla gestione dello stato.

Le tre componenti principali del pattern MVVM sono: Model, ViewModel e View, come rappresentato nello schema seguente.



- `View`: responsabile dell'interfaccia utente. La View si occupa della presentazione grafica e dell'interazione con l'utente, ma non contiene logica applicativa. Essa riceve dati dal ViewModel e si aggiorna automaticamente tramite meccanismi reattivi di gestione dello stato.

- `ViewModel`: funge da intermediario tra View e Model. Ha il compito di gestire lo stato dell'interfaccia utente e di elaborare la logica che collega la UI ai dati. In Android ciò avviene, ad esempio, tramite StateFlow, mentre in Flutter si possono utilizzare strumenti come Riverpod. Il ViewModel espone dati osservabili che la View può ascoltare per reagire a ogni cambiamento.

- `Model`: rappresenta i dati e la logica di business dell'applicazione. In questa architettura è suddiviso in tre componenti:

  – *Repository*: funge da punto centrale per l'accesso ai dati. Riceve richieste dal ViewModel e decide se prelevare i dati da una sorgente locale o remota.

  – *LocalDataSource*: gestisce l'accesso a dati persistiti localmente, ad esempio in un database (es. Room per Android)

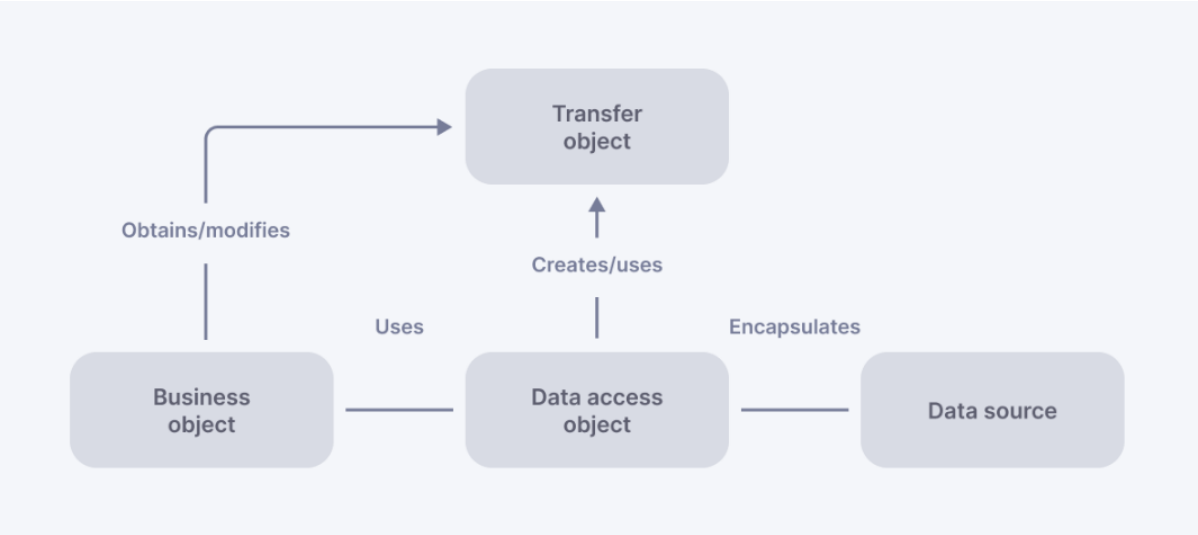
  – *RemoteDataSource*: si occupa della comunicazione con servizi remoti, come API REST, inviando e ricevendo dati via rete.

  Nonostante la tipica organizzazione, il Model del nostro progetto sarà composto solamente da Repository e RemoteDataSource.

4.2 Descrizione dell'architettura DAO

L'applicazione utilizza un'architettura dati basata sul pattern DAO (Data Access Object), che consente di separare in modo pulito la logica applicativa dalla logica di accesso ai dati. Questa struttura favorisce la manutenibilità, la testabilità e la scalabilità dell'intero sistema.

Le quattro componenti principali del pattern DAO sono: Data Source, Data Access Object, Transfer Object e Business Object, come rappresentato nello schema seguente.



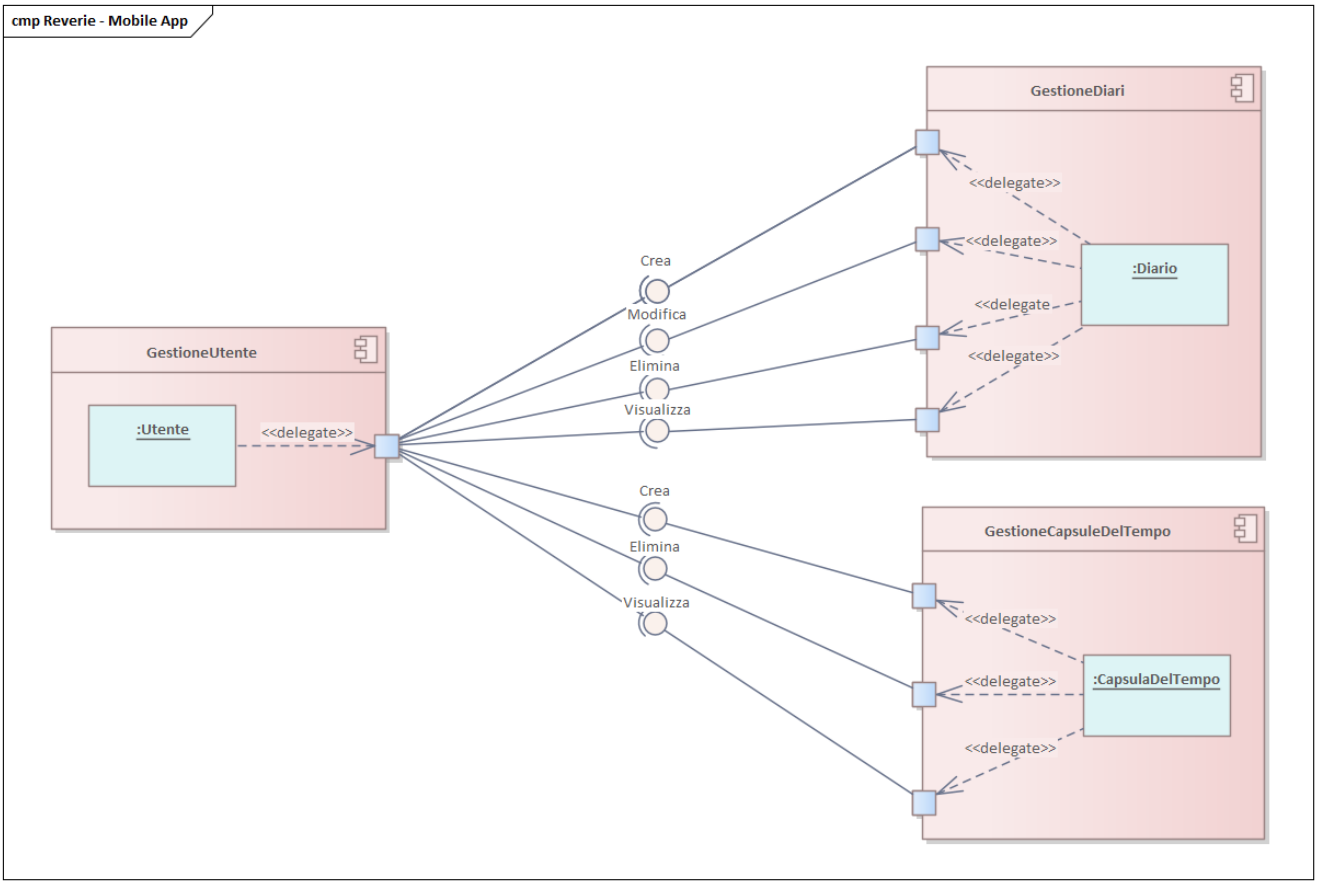
- `Data Source`: è la sorgente dei dati persistenti. Può essere un database locale (es. Room), un'API REST o un sistema remoto (es. Firebase). Nel nostro progetto viene utilizzato Firebase.
- `Transfer Object (TO)`: sono gli oggetti contenenti dati che viaggiano tra i vari livelli dell'applicazione. Nella nostra architettura, corrispondono alle classi model (come Diary, e TimeCapsule) che definiscono le proprietà e i metodi di serializzazione/deserializzazione per comunicare con il DAO.
- `Data Access Object (DAO)`: questo livello si occupa esclusivamente dell'accesso ai dati, incapsulando tutte le chiamate al database e isolando l'app dalla logica specifica del sistema di persistenza. Nel nostro progetto, questo ruolo è svolto dallo Storage Service, che implementa metodi che si interfacciano direttamente con il database.
- `Business Object (BO)`: questa componente contiene la logica di business dell'applicazione. Decide quando, come e in che forma devono essere utilizzati i dati. Nel nostro caso, la classe Repository agisce come Business Object, in quanto:
  - Definisce metodi di uso generale, astratti rispetto al tipo di database.
  - Coordina l'accesso ai dati tra ViewModel e Storage Service.

4.3 Firebase come Remote Data Source

Il nostro progetto utilizza Firebase Firestore per memorizzare i dati della nostra applicazione e, tramite Firebase Authentication, viene gestita l'autenticazione degli utenti. In Firebase vengono distinte le seguenti cartelle di progetto:

- `diaries`: ciascun documento di questa cartella rappresenta un diario di un utente.
- `diaryCovers`: ciascun documento identifica una delle cover che l'utente può scegliere come copertina per il proprio diario, composto da un url ottenuto dopo aver inserito l'immagine della cover all'interno di un bucket su Supabase.
- `diaryImages`: ciascun documento identifica un'immagine precisa all'interno di un diario, con la relativa pagina e offset. Anche questo presenta un url ottenuto dopo l'inserimento dell'immagine nel bucket.
- `pages`: ciascun documento indica un racconto quotidiano nel diario.
- `subPages`: ciascun documento indica una pagina di un racconto quotidiano e questa cartella serve prettamente per la gestione del posizionamento delle immagini al'interno del diario.
- `timeCapsules`: ciascun documento rappresenta una capsula del tempo di un utente.
- `users`: ciscun documento rappresenta l'account di un utente.

4.4 Diagramma delle Componenti



4.5 Mockup

Figure 4.1: Pagina dei Diari

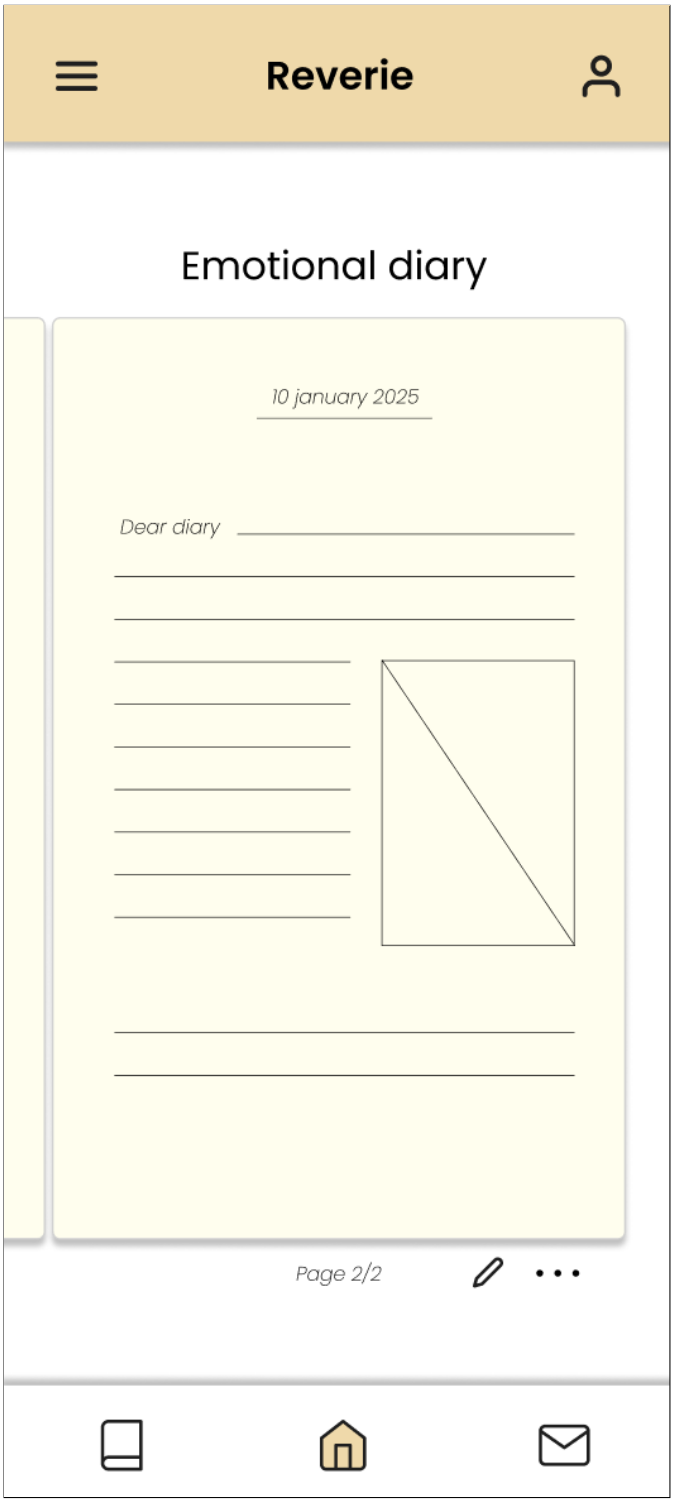


Figure 4.2: Pagina di Visualizzazione del Diario

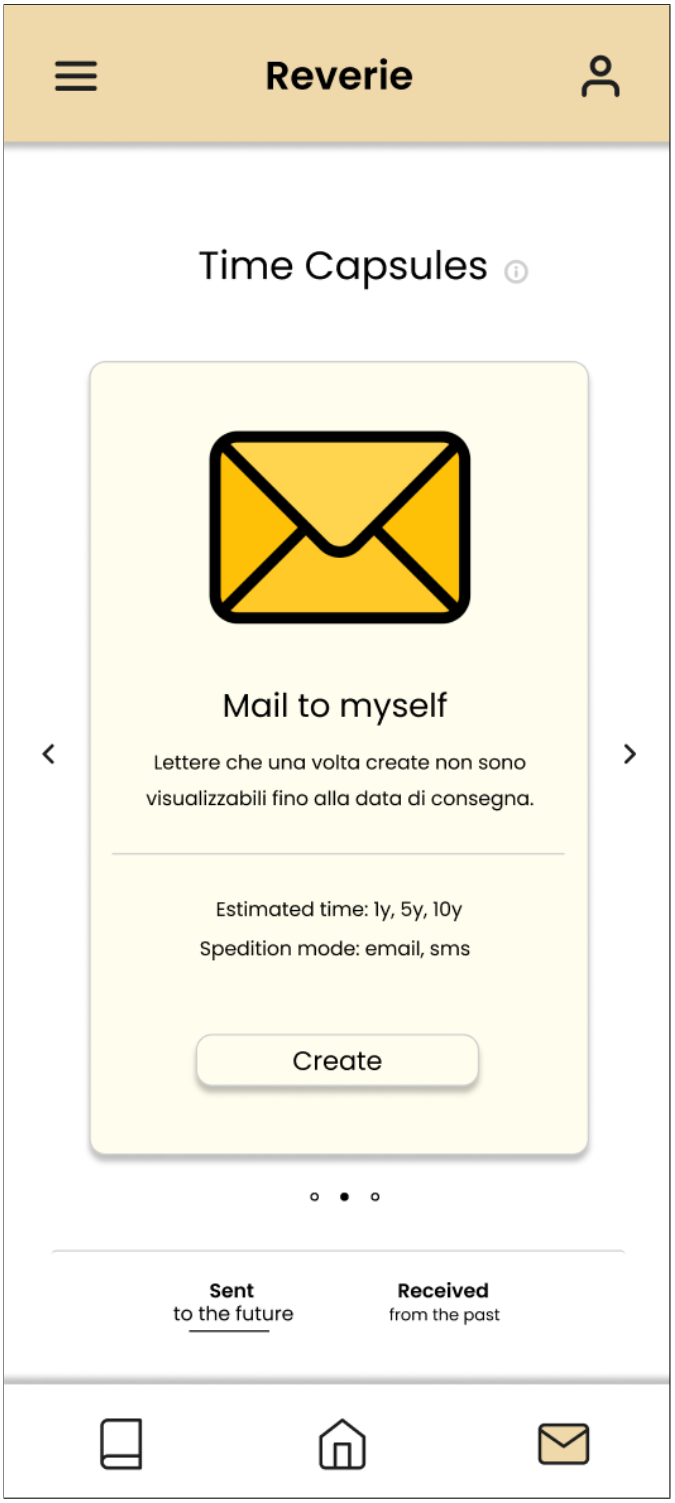


Figure 4.3: Pagina delle Capsule del Tempo

5. Implementazione in Kotlin

Per sviluppare la nostra applicazione Android in Kotlin abbiamo deciso di adottare Jetpack Compose per la gestione dell'interfaccia utente. Questa scelta nasce dall'esigenza di creare un'app moderna, reattiva e coerente con le più recenti linee guida di design Android. La scelta progettuale di adottare Jetpack Compose al posto del tradizionale approccio basato su XML è stata motivata da diversi fattori tra cui:

- `Dichiaratività`: Compose consente di descrivere l'interfaccia utente in modo dichiarativo, rendendo il codice più leggibile e allineato alla logica dell'applicazione. Ogni stato dell'interfaccia è direttamente legato allo stato dell'app, riducendo la complessità del ciclo di vita delle UI.
- `Maggiore integrazione con lo state management`: Compose è progettato per funzionare nativamente con l'architettura MVVM.

5.1 Struttura del File System del Progetto

L'adozione dell'architettura MVVM ha portato a una struttura ben organizzata delle cartelle del progetto.

Struttura delle cartelle principali

- `data`: contiene tutti gli elementi legati alla gestione e al recupero dei dati.
  - *model*: definisce le classi dei modelli utilizzati all'interno dell'applicazione. Queste classi sono progettate in maniera tale da poter essere salvate nel database (hanno quindi dei metodi appositi per la conversione da/a JSON).
  - *repository*: implementa la logica di accesso ai dati. I repository fungono da intermediari tra la sorgente dati (remota) e il ViewModel, incapsulando la logica di business e centralizzando il flusso dei dati.
- `navigation`: gestisce la navigazione all'interno dell'app tramite il sistema di routing di Jetpack Compose. Contiene le definizioni delle rotte, gli argomenti e le transizioni tra gli screen.
- `ui`: raccoglie tutti gli elementi relativi all'interfaccia utente.
  - *components*: include componenti UI riutilizzabili e modulari progettati per essere facilmente integrabili in più schermate.
  - *screens*: contiene le singole schermate dell'applicazione. Ogni screen incapsula un flusso logico e visivo specifico dell'app, collegato a un ViewModel.
  - *theme*: ospita i file responsabili della definizione dello stile grafico globale dell'app, sfruttando il sistema di theming di Jetpack Compose.

- `viewmodel`: contiene i ViewModel associati alle diverse schermate dell'app. I ViewModel gestiscono lo stato dell'interfaccia utente, reagiscono agli eventi dell'utente e orchestrano il recupero dei dati dai repository.

File principali a livello di root

- `MainActivity.kt`: punto di ingresso dell'app. Qui avviene il set-up iniziale dei componenti di Jetpack Compose, si crea l'ambiente ComposeView e si avvia il sistema di navigazione.

- `StorageService.kt`: gestisce l'interazione con il database (Firebase). Definisce i metodi per effettuare operazioni CRUD, permettendo lo scambio di dati tra database e repository.

File di configurazione

- `build.gradle di progetto`: questo file rappresenta la configurazione globale del progetto. Definisce i plugin comuni utilizzati nei vari moduli, come quelli per Android, Kotlin, Compose, Dagger/Hilt e Google Services.

- `build.gradle di modulo`: è il file di configurazione principale per il modulo dell'applicazione. Qui vengono definite le impostazioni di compilazione (come versione SDK e versione Java), attivati i plugin necessari (come Compose, Dagger/Hilt, Google Services, etc.) e dichiarate tutte le dipendenze utilizzate nel progetto. Ad esempio, sono presenti librerie per Firebase, serializzazione, gestione delle immagini, e mocking. Questo file determina il comportamento dell'intero modulo durante la fase di sviluppo, compilazione e testing.

5.2 Approfondimenti sul codice

Per quanto riguarda lo sviluppo in Jetpack Compose, come detto precedentemente, abbiamo seguito un approccio classico, usando il pattern MVVM. Tra le librerie utilizzate nel progetto, riteniamo importante trattarne due nello specifico: `Dagger/Hilt` e `TextFlow`.

Per quanto riguarda `Dagger/Hilt`, si tratta di una libreria molto diffusa per la dependency injection in ambiente Jetpack Compose. Il suo principale vantaggio è la drastica riduzione del codice boilerplate necessario per configurare l'iniezione delle dipendenze. È sufficiente definire una classe che fornisca i vari provider delle dipendenze, mentre il resto del processo viene gestito automaticamente tramite annotazioni, applicate direttamente ai costruttori o ai punti di iniezione.

Relativamente a `TextFlow`, si tratta di una libreria focalizzata sul rendering del testo attorno a una o più immagini, rendendo possibile la visualizzazione delle pagine di diario all'interno dell'app. Tuttavia, essendo una libreria di nicchia e con una piccola community alle spalle, presenta alcune limitazioni, in particolare la mancanza di reattività durante lo spostamento delle immagini. Per aggirare questo problema, è stato necessario implementare diversi workaround tramite porzioni di codice utilizzate unicamente per forzare la ricomposizione dell'interfaccia.

5.2.1 Model

- `Diary`: rappresenta un diario ed implementa l'interfaccia Parcelable per consentire il passaggio degli oggetti tra componenti dell'applicazione. Tra le sue proprietà principali vi sono l'id

del diario stesso, l'identificativo dell'utente a cui appartiene, oltre a titolo, descrizione e data di creazione. Inoltre vi è l'identificativo dell'immagine di copertina del diario e la lista degli identificativi delle pagine di cui è composto.

La classe effettua l'override di un metodo dell'interfaccia *Parcelable* per escludere l'attributo *stability*, sempre di *Parcelable*, dalla serializzazione degli oggetti Diary.

- `DiaryCover`: rappresenta una copertina di un diario. Presenta un proprio identificativo, con un nome e l'url dell'immagine.

- `DiaryImage`: rappresenta un'immagine del diario. Oltre ad avere un proprio identificativo e l'url dell'immagine, presenta l'identificativo della pagina di diario in cui l'immagine è collocata. Inoltre vengono memorizzate informazioni per lo specifico posizionamento dell'immagine nella pagina.

- `DiaryPage`: rappresenta un racconto quotidiano di un diario, ossia un insieme di pagine relative ad uno stesso giorno. Tra gli attributi principali, oltre al proprio identificativo e l'identificativo del diario a cui appartiene, vi sono il contenuto testuale del racconto e una lista degli identificativi delle singole pagine che lo compongono.

- `DiarySubPage`: rappresenta la singola pagina di un racconto quotidiano del diario. Presenta un proprio identificativo e l'identificativo del racconto quotidiano a cui appartiene. Inoltre presenta *contentEndIndex*, che indica l'indice dell'ultimo carattere del racconto quotidiano visualizzabile nella pagina, *refreshCounter*, che è una variabile fittizia per permettere la ricomposizione del *TextFlow*, e *testOverflow*, che indica in quale step del calcolo dell'overflow ci troviamo.

- `Email`: rappresenta l'indirizzo email di un utente. Presenta come identificativo la mail stessa e come campo lo username associato alla email. Questo modello è stato realizzato per questioni di efficienza per eseguire un controllo di unicità del campo direttamente lato server. Tuttavia questa scelta introduce delle ridondanze nel database.

- `TimeCapsule`: rappresenta una capsula del tempo. Presenta un proprio identificativo e l'identificativo dell'utente creatore. Inoltre presenta un titolo, una descrizione , la data di consegna e di creazione. Fra gli attributi vi sono anche la lista delle email, dei numeri di telefono e degli utenti a cui la capsula deve essere spedita.

- `User`: rappresenta un utente della piattaforma. Questo modello ha un proprio identificativo, un campo email, username, nome e cognome. Inoltre vi sono la lista degli identificativi dei diari e delle capsule del tempo che appartengono all'utente. Quest'ultima viene divisa in "capsule del tempo inviate" e "capsule del tempo ricevute" in modo tale da distinguere le lettere inviate da quelle ricevute da altri utenti.

- `Username`: rappresenta lo username di un utente. Per questo modello vale lo stesso ragionamento fatto per il modello *Email*.

5.2.2 Repository

I repository, come accennato in precedenza, hanno il compito di disaccoppiare l'accesso al database dalla logica contenuta nei *ViewModel*. I tre repository implementati condividono alcune caratteristiche comuni: ciascuno è composto da un'interfaccia e dalla relativa implementazione concreta che riceve lo *StorageService*, tramite *dependency injection*, utilizzato per interagire con il database. Inoltre, riceve anche altri repository quando è necessario accedere a dati esterni al proprio ambito.

Ogni repository implementa metodi *get*, che accettano un identificativo come parametro e sollevano un'eccezione in caso di errore; metodi *save*, che ricevono un'istanza di modello da salvare; metodi *update*, per gestire modifiche a un'istanza già esistente; infine, metodi *delete*, che si occupano della rimozione di un'istanza.

Nei metodi *save* e *delete*, inoltre, è necessario gestire manualmente i riferimenti tra documenti, tipici di un database non relazionale, sincronizzando ad esempio l'identificativo del nuovo elemento con le liste di ID contenute in altri documenti.

- `DiaryRepository`: Contiene tutti i metodi necessari per eseguire operazioni CRUD sui modelli relativi ai diari. Utilizza lo *UserRepository* per accedere a dati utente nei metodi *getUserDiaries* e *saveDiary*.

- `TimeCapsuleRepository`: Contiene tutti i metodi necessari per eseguire operazioni CRUD sui modelli relativi alle capsule del tempo. Utilizza lo *UserRepository* per accedere a dati utente nei metodi *getUserSentTimeCapsules*, *getUserReceivedTimeCapsules* e *saveTimeCapsule*.

- `UserRepository`: Contiene tutti i metodi necessari per eseguire operazioni CRUD sui modelli relativi agli utenti. Utilizza il *DiaryRepository* per accedere a dati dei diari nel metodo *deleteUser*. Inoltre utilizza anche FirebaseAuth per i metodi *authenticate*, *createAccount* e *sendPasswordResetEmail*.

5.2.3 Navigation

- `CustomNavHost`: definisce le rotte dell'applicazione, le quali sono divise in quattro aree: *AuthenticationRoutes*, *DiaryRoutes*, *ProfileRoutes* e *TimeCapsuleRoutes*. Inoltre, per ciascuno screen di ogni rotta, definisce il contenuto di funzioni lambda utili per la navigazione lungo lo stack.

- `AuthenticationRoutes`, `DiaryRoutes`, `ProfileRoutes`, `TimeCapsuleRoutes`: rappresentano file che contengono object o data class per definire univocamente le rotte. Nel caso di data class vengono definiti anche i parametri da passare alla rotta.

5.2.4 Screen

Lo screen osserva lo stato esposto dal *ViewModel* e aggiorna la UI in modo reattivo senza contenere logica di business o gestione dati. In caso di caricamento, mostra un indicatore di progresso, mentre in caso di errore visualizza un messaggio testuale.

- `AllDiariesScreen`: consente la visualizzazione e la gestione dei diari di un utente. Mostra un *HorizontalPager*, con le *Card* rappresentanti i singoli diari, ognuna contenente la copertina, titolo, descrizione e pulsanti di modifica e cancellazione. Sotto il pager, un *ButtonBar* permette di scegliere se visualizzare le informazioni del diario (come data di creazione e numero di pagine) o se visualizzare una galleria di immagini contenute al suo interno. Supporta anche un dialogo di conferma per la cancellazione di un diario e un *FloatingActionButton* per la creazione di un nuovo diario.

- `AllTimeCapsulesScreen`: consente la visualizzazione di una lista di capsule del tempo suddivise in tre categorie: *Scheduled*, *Sent* e *Received*. Lo screen è composto da una *LazyColumn*. Esso presenta una *Card* al cui interno vi è la descrizione di cos'è una capsula del tempo e la relativa immagine. Inoltre, vi è una *ButtonBar* utile per selezionare la categoria di capsule da visualizzare. Per ciascuna sezione viene mostrata la lista delle capsule che ne fanno parte, ognuna avente un'immagine, titolo, numero di destinatari, data di scadenza e di creazione. Le capsule nella categoria *Scheduled* presentano anche un'icona per aprire il dialogo di conferma cancellazione. È inoltre presente un *FloatingActionButton* in alto a destra per navigare alla schermata di creazione di una nuova capsula.

- `CreateTimeCapsuleScreen`: consente all'utente di creare una nuova capsula del tempo. L'utente può inserire il titolo e il contenuto della capsula del tempo. Lo screen permette all'utente di inserire lo username, l'email o il numero di telefono dei destinatari, i quali, dopo l'inserimento, verranno mostrati in una sezione apposita dove l'utente può decidere se eliminarli. Alla pressione del pulsante di conferma, viene inviata la richiesta di validazione e creazione della capsula. In caso di successo, viene creata una *TimeCapsule*, altrimenti vengono mostrati i messaggi di errore generati sotto i corrispondenti campi.

- `EditDiaryPageScreen`: permette la modifica del contenuto di un racconto quotidiano del diario selezionato dall'utente. Questo screen presenta un campo multilinea con il contenuto testuale modificabile dall'utente e un bottone di conferma modifica.

- `EditDiaryScreen`: permette la personalizzazione di un diario. L'utente può modificare il titolo del diario e la sua descrizione, inoltre può scegliere l'immagine di copertina del diario. Questo screen viene utilizzato sia nel caso in cui l'utente selezioni il pulsante di modifica di un diario già esistente che nel caso l'utente scelga di crearne uno nuovo.

- `EditProfileScreen`: permette all'utente di modificare il proprio profilo. I campi modificabili sono il proprio username, nome e cognome. Nel caso in cui ci sono errori verranno mostrati appositi messaggi sotto i campi corrispondenti.

- `LoginScreen`: permette all'utente di effettuare il login. L'utente deve inserire la sua email e la password per poter accedere. Nel caso di errori, ad esempio se le credenziali sono errate, vengono mostrati messaggi di errore sotto gli appositi campi.

- `MainScreen`: definisce lo scheletro dell'interfaccia dell'app, tramite uno *Scaffold* con una *TopBar* ed una *BottomBar* personalizzate, e della navigazione grazie al *NavController* e ad un *NavHost* personalizzato.

- `ProfileScreen`: permette all'utente di visualizzare le informazioni relative al proprio profilo. Inoltre vi è un pulsante che permette all'utente di spostarsi nello screen di modifica del profilo.

- `ResetPasswordScreen`: permette all'utente di eseguire il reset della password del proprio account. Questo screen presenta un campo in cui l'utente deve inserire l'email alla quale vuole ricevere il link per il reset della password dell'account.

- `SignupScreen`: permette all'utente di creare un proprio account sulla piattaforma. Questo screen presenta dei campi in cui l'utente deve inserire il proprio username, nome e cognome oltre ad indirizzo email, password e password di conferma. Nel caso di errori nel form, vengono visualizzati i relativi messaggi sotto gli appositi campi.

- `ViewDiaryScreen`: permette all'utente di mostrare il diario da lui selezionato. Per la visualizzazione delle pagine del diario, è stata utilizzata una *LazyRow*, in modo tale che l'utente possa scorrerle orizzontalmente. Oltre alle pagine, lo screen presenta un bottone per l'inserimento delle immagini nella *DiarySubPage* visualizzata e un bottone per far spostare l'utente nello screen di modifica del testo della pagina.

- `ViewTimeCapsuleScreen`: permette all'utente di visualizzare le informazioni relative ad una capsula del tempo. Lo screen permette di visualizzare il titolo della capsula, l'eventuale contenuto, nel caso la data di consegna della capsula sia passata, la data di creazione e, nel caso l'utente sia il mittente della lettera, anche i destinatari a cui è stata inviata.

5.2.5 Components

- `ButtonBar`: componente che mostra una barra di pulsanti orizzontale. Prende in input lo stato attuale, l'elenco dei possibili stati, una lambda per l'aggiornamento dello stato e una funzione per ottenere il nome visualizzato di ciascun elemento. Il pulsante corrispondente allo stato selezionato viene evidenziato.

- `ConfirmDelete`: componente che mostra un *AlertDialog* per confermare l'eliminazione di un elemento. Accetta un titolo, un messaggio descrittivo, un callback per l'annullamento e uno per la conferma dell'eliminazione. Fornisce due pulsanti: "Cancella" (in rosso) e "Annulla".

- `ContentTextField`: componente che mostra un campo di testo con etichetta. Accetta il valore corrente, una lambda per l'aggiornamento, un'etichetta descrittiva e un *Modifier* opzionale. È disponibile anche in una versione estesa che mostra un messaggio di errore al di sotto del campo, tramite il componente *ErrorField*.

- `CustomBottomBar`: componente che implementa una barra di navigazione inferiore personalizzata. Mostra due voci di navigazione principali (diari e capsule del tempo), ciascuna rappresentata da un'icona e una label. La funzione utilizza il *NavController* per gestire la navigazione e mantenere lo stato tra le schermate. L'elemento selezionato viene evidenziato.

- `CustomTopBar`: componente che definisce una barra superiore dell'applicazione. Mostra il nome dell'app come titolo allineato a sinistra e un'icona profilo sulla destra. Al clic sull'icona si navigherà alla route per la visualizzazione del profilo utente.

- `ErrorField`: componente che mostra il testo dato in input in rosso con font small. È pensata per visualizzare messaggi di errore sotto campi di input.

- `PasswordField`: componente che visualizza un campo di input per password, con testo oscurato. Accetta il valore corrente, una lambda per l'aggiornamento, un'etichetta e un `Modifier` opzionale. È disponibile anche in una versione estesa che mostra un messaggio di errore al di sotto del campo, tramite il componente *ErrorField*.

- `PhoneNumber`: componente che mostra un campo di input per numeri di telefono. Accetta il valore attuale, una lambda per l'aggiornamento, un modificatore e un'icona opzionale. È disponibile anche una variante che visualizza un messaggio di errore sotto il campo tramite il componente *ErrorField*. Supporta la formattazione dinamica grazie a `PhoneNumberVisualTransformation`, che utilizza la libreria *libphonenumber* di Google per applicare il formato corretto in base al paese dell'utente. Gestisce inoltre il corretto posizionamento del cursore tra il testo originale e quello formattato.

- `SingleLineField`: componente che mostra un campo di testo a riga singola con etichetta e supporto opzionale per un'icona a destra. Accetta il valore corrente, una lambda per l'aggiornamento, un'etichetta, un modificatore e un'icona trailing. È disponibile anche in una versione estesa che mostra un messaggio di errore al di sotto del campo, tramite il componente *ErrorField*.

5.2.6 Theme

- `Color.kt`: questo file include configurazioni per i colori dell'app.

- `Theme.kt`: questo file definisce lo stile visivo generale dell'app.

- `Type.kt`: questo file include configurazioni per la tipografia dell'app.

5.2.7 ViewModel

I ViewModel si occupano di preparare i dati da visualizzare nella UI e di fornire metodi per modificarli. Ogni ViewModel mantiene uno stato contenente i dati, osservabile dalla UI tramite uno *StateFlow*. All'avvio, il ViewModel carica dal database tutti i dati necessari e li memorizza nello stato. Lo stato gestito da un ViewModel ha solitamente tre possibili valori:

- `Loading`: durante il caricamento iniziale;

- `Success`: in caso di caricamento avvenuto con successo;

- `Error`: in caso di errore durante il recupero dei dati.

Ogni modifica dello stato avviene attraverso metodi dedicati, che garantiscono un aggiornamento controllato e reattivo.

- `AllDiariesViewModel`: gestisce lo stato di *AllDiariesScreen*. Alla sua creazione, carica nello stato i diari dell'utente, recuperando anche le relative copertine e immagini. Il ViewModel contiene la logica che permette la navigazione tra i vari diari, l'eliminazione degli stessi

(con gestione del dialog di conferma) e la possibilità di mostrare la sezione info o la sezione immagini. Inoltre, quando si ritorna dalla navigazione a *EditDiaryRoute*, verranno aggiornati i dati del diario selezionato o verrà aggiunto il nuovo diario creato tramite degli appositi metodi. Il *PagerState* per la navigazione dei diari è configurato per simulare uno scorrimento ciclico infinito.

- `AllTimeCapsulesViewModel`: gestisce lo stato di *AllTimeCapsulesScreen*. Alla sua creazione, carica nello stato le time capsule associate all'utente corrente. Il ViewModel contiene la logica che permette permette la navigazione tra le varie tipologie di capsule del tempo e l'eliminazione delle time capsule programmate (con gestione del dialog di conferma). Inoltre, quando si ritorna dalla navigazione a *CreateTimeCapsulesRoute*, verrà aggiunta la nuova capsula allo stato tramite il metodo *addNewTimeCapsule*.

- `CreateTimeCapsuleViewModel`: gestisce lo stato di CreateTimeCapsuleScreen. Questo ViewModel contiene la logica per la validazione dei campi del form e per l'inserimento ed eliminazione dei destinatari.

- `EditDiaryPageViewModel`: gestisce lo stato di *EditDiaryPageScreen*. Alla sua creazione, carica nello stato i dati della pagina da modificare. Il ViewModel permette la modifica del contenuto della pagina.

- `EditDiaryViewModel`: gestisce lo stato di *EditDiaryScreen*. Può essere utilizzato sia per la creazione che per la modifica di una diario. Alla sua creazione, carica nello stato i dati dell'eventuale diario da modificare. Il ViewModel contiene la logica che permette la modifica del titolo, contenuto e copertina del diario, con validazione degli input.

- `EditProfileViewModel`: gestisce lo stato di *EditProfileScreen*. Alla sua creazione, carica nello stato i dati del profilo da modificare. Il ViewModel contiene la logica che permette la modifica dello username, del nome e del cognome, con validazione degli input.

- `LoginViewModel`: gestisce lo stato di *LoginScreen*. Il ViewModel contiene la logica che permette di effettuare il login con validazione reattiva degli input.

- `MainViewModel`: gestisce lo stato di *MainScreen*. Il ViewModel contiene dei metodi per controllare se l'utente è autenticato, per effettuare il logout e per ottenere l'id utente.

- `ProfileViewModel`: gestisce lo stato di *ProfileScreen*. Il ViewModel contiene la logica che permette la visualizzazione dei dati del profilo utente, tra cui username, nome e cognome.

- `ResetPasswordViewModel`: gestisce lo stato di *ResetPasswordScreen*. Il ViewModel permette di effettuare l'invio dell'email contenente il link per resettare la password.

- `SignupViewModel`: gestisce lo stato di *SignupScreen*. Il ViewModel contiene la logica che permette di effettuare il signup con validazione reattiva degli input. In particolare viene effettuato il controllo per la disponibilità dello username e dell'email inseriti.

- `ViewDiaryViewModel`: gestisce lo stato di *ViewDiaryScreen*. Alla sua creazione, carica nello stato le pagine del diario selezionato e le relative immagini. Il ViewModel contiene la logica

che permette la navigazione tra le varie pagine del diario, l'eliminazione di una pagina (con gestione del dialog di conferma), modifica di una pagina e gestione delle immagini (inserimento, eliminazione e spostamento).

Questo ViewModel presenta una maggiore complessità rispetto agli altri a causa delle limitazioni della libreria *TextFlow* già descritte in precedenza, e per questo il suo funzionamento merita una spiegazione più esaustiva.

Una delle principali criticità è la necessità di forzare manualmente la ricomposizione dell' interfaccia ogni volta che il testo viene modificato o che un'immagine viene spostata. Per gestire questa esigenza, abbiamo introdotto un contatore interno a ciascuna *SubPage*, che viene incrementato a ogni modifica. Questo contatore viene poi esposto nello *Screen* tramite un elemento *Text* invisibile, in modo da attivare la ricomposizione di Compose.

Tuttavia, questa soluzione da sola non è sufficiente a garantire un aggiornamento corretto dell'interfaccia. Infatti, per come è strutturata la visualizzazione delle pagine, è necessario calcolare manualmente l'indice di inizio e fine del testo da visualizzare per ogni *SubPage*.

Per eseguire questo calcolo, è fondamentale che tutte le sotto-pagine appartenenti allo stesso racconto quotidiano siano visibili contemporaneamente. Questo permette al *TextFlow* di aggiornare correttamente il layout e calcolare l'offset, ovvero il numero di caratteri effettivamente renderizzati in ciascuna pagina.

Tuttavia, l'uso di una *LazyRow* — scelta necessaria per motivi di performance — impedisce che tutte le pagine vengano visualizzate contemporaneamente. Per aggirare questo vincolo, abbiamo implementato, utilizzando una seconda *Row* invisibile, una duplicazione delle pagine dello stesso racconto quotidiano per cui non è stato ancora calcolato l'offset.

Questa soluzione, pur aumentando leggermente il carico sull'interfaccia, risolve il problema dell'aggiornamento degli indici.

- `ViewTimeCapsuleViewModel`: gestisce lo stato di *ViewTimeCapsuleScreen*. Alla sua creazione, carica nello stato le time capsule associate all'utente. Il ViewModel contiene la logica che permette la navigazione tra le varie tipologie di capsule del tempo e l'eliminazione delle time capsule programmate (con gestione del dialog di conferma). Inoltre, quando si ritorna dalla navigazione a *CreateTimeCapsulesRoute*, verrà aggiunta la nuova capsula allo stato tramite il metodo *addNewTimeCapsule*.

5.3 Descrizione unit tests

I seguenti script di codice rappresentano un insieme di unit test utilizzati per garantire il corretto funzionamento di alcune funzionalità della nostra applicazione. Sono state testate le funzionalità principali del sistema quali: la visualizzazione dei diari e delle capsule del tempo di un utente. Grazie a questi test è stato possibile verificare che l'applicazione si comporti come previsto oltre ad individuare eventuali bug in fase di sviluppo.

5.3.1 Mock repository

Per simulare l'applicazione durante i test, è necessario predisporre un insieme limitato di dati. Per questo scopo, vengono creati dei repository "fake" incaricati di fornire i dati necessari.

`FakeDiaryRepository`, `FakeTimeCapsuleRepository` e `FakeUserRepository` sono classi che vanno a fare rispettivamente il mock di *DiaryRepository*, *TimeCapsuleRepository* e *UserRepository* tramite un override di tutti i metodi forniti dall'interfaccia. In questo modo, i repository non hanno più bisogno di usare i metodi all'interno di *StorageService* in quanto i dati da utilizzare vengono generati direttamente in fase di creazione.

5.3.2 Mock ServiceModule

Infine, è necessario fare il mock del *ServiceModule* in quanto sarà lui che andrà a fornire i dati ai FakeRepository appena definiti all'applicazione di test. Tramite la libreria testing di dagger hilt, si va ad annotare *TestServiceModule* nel seguente modo:

```
@Module
@TestInstallIn(
    components = [SingletonComponent::class],
    replaces = [ServiceModule::class]
)
object TestServiceModule {
    ...
}
```

A questo punto la nostra applicazione è pronta per l'esecuzione dei test utilizzando un dataset fittizio.

5.3.3 Classe AllDiariesScreenTest

Test displaysDiaryTitleAndRespondsToFabClick

Il test verifica che, al caricamento iniziale della schermata AllDiariesScreen, venga mostrato correttamente un diario con il titolo "Test Diary". Inoltre, controlla che il pulsante flottante (FAB) per la creazione di un nuovo diario sia visibile e risponda correttamente al clic da parte dell'utente.

```
@Test
fun displaysDiaryTitleAndRespondsToFabClick() {
    composeTestRule.activity.setContent {
        AllDiariesScreen(
            onNavigateToDiary = {},
            onNavigateToEditDiary = {},
            onNavigateToCreateDiary = {}
        )
    }
    val fabContentDescription = composeTestRule.activity.getString(R.string.create_diary)
    composeTestRule
        .onNodeWithText("Test Diary")
        .assertIsDisplayed()
    composeTestRule
        .onNodeWithContentDescription(fabContentDescription)
        .assertIsDisplayed()
        .performClick()
}
```

Test showsImageTabOnImageButtonClick

Il test simula il clic sul pulsante "Immagini" all'interno della schermata dei diari. Dopo il clic, il test attende il caricamento delle immagini e verifica che almeno un'immagine venga effettivamente mostrata, tramite il controllo della descrizione contenuta nei metadati di accessibilità (content description).

```
@Test
fun showsImageTabOnImageButtonClick() {
    composeTestRule.activity.setContent {
        AllDiariesScreen(
            onNavigateToDiary = {},
            onNavigateToEditDiary = {},
            onNavigateToCreateDiary = {},
        )
    }
    composeTestRule
        .onNodeWithText("Test Diary")
        .assertIsDisplayed()
    val imagesButtonText = composeTestRule.activity.getString(R.string.images)
    composeTestRule
        .onNodeWithText(imagesButtonText)
        .assertIsDisplayed()
        .performClick()
    val imageContentDesc = composeTestRule.activity.getString(R.string.image)
    composeTestRule.waitUntil(
        condition = {
            composeTestRule
                .onAllNodesWithContentDescription(imageContentDesc)
                .fetchSemanticsNodes()
                .isNotEmpty()
        },
        timeoutMillis = 5_000
    )
    composeTestRule
        .onAllNodesWithContentDescription(imageContentDesc)
        .onFirst()
        .assertIsDisplayed()
}
```

Test displaysMultipleDiariesAndHandlesPagerSwipe

Il test testa la navigazione tra più diari tramite gesture di swipe orizzontale. Parte dal diario iniziale "Test Diary", poi simula due swipe a sinistra e verifica la visualizzazione dei diari successivi: "Second Diary" e "Third Diary". Questo garantisce che il pager funzioni correttamente nella gestione di più elementi.

```
@Test
fun displaysMultipleDiariesAndHandlesPagerSwipe() {
    composeTestRule.activity.setContent {
        AllDiariesScreen(
            onNavigateToDiary = {},
            onNavigateToEditDiary = {},
            onNavigateToCreateDiary = {}
        )
    }
    composeTestRule.onNodeWithText("Test Diary").assertIsDisplayed()
    composeTestRule
        .onNodeWithText("Test Diary")
        .performTouchInput { swipeLeft() }
    composeTestRule.onNodeWithText("Second Diary").assertIsDisplayed()
    composeTestRule
        .onNodeWithText("Second Diary")
        .performTouchInput { swipeLeft() }
    composeTestRule.onNodeWithText("Third Diary").assertIsDisplayed()
}
```

Test switchingToInfoTabDisplaysCreationDateAndPageNumber

Il test simula il passaggio alla sezione "Info" di un diario. Dopo il clic sul tab corrispondente, il test verifica che vengano visualizzate le etichette relative alla data di creazione e al numero di pagina del diario, confermando che le informazioni testuali vengano mostrate correttamente all'utente.

```
@Test
fun switchingToInfoTabDisplaysCreationDateAndPageNumber() {
    composeTestRule.activity.setContent {
        AllDiariesScreen(
            onNavigateToDiary = {},
            onNavigateToEditDiary = {},
            onNavigateToCreateDiary = {}
        )
    }
    val infoText = composeTestRule.activity.getString(R.string.info)
    val creationDateLabel = composeTestRule.activity.getString(R.string.creation_date) + ":"
    val pageNumberLabel = composeTestRule.activity.getString(R.string.page_number) + ":"
    composeTestRule
        .onNodeWithText(infoText, ignoreCase = true)
        .performClick()
    composeTestRule.onNodeWithText(creationDateLabel).assertIsDisplayed()
    composeTestRule.onNodeWithText(pageNumberLabel).assertIsDisplayed()
}
```

5.3.4 Classe AllTimeCapsulesScreenTest

Test displaysScheduledCapsulesByDefault

Il test verifica che, al primo avvio dello schermo AllTimeCapsulesScreen, vengano mostrate le capsule temporali programmate (cioè quelle in programma per il futuro). Il test controlla che una capsula denominata "Future Capsule" sia visibile all'utente.

```
@Test
fun displaysScheduledCapsulesByDefault() {
    composeTestRule.activity.setContent {
        AllTimeCapsulesScreen(
            onNavigateToViewTimeCapsule = {_, _ -> },
            onNavigateToCreateTimeCapsule = {},
        )
    }
    composeTestRule
        .onNodeWithText("Future Capsule")
        .assertIsDisplayed()
}
```

Test switchesToSentCapsulesOnSentButtonClick

Il test simula il clic sul pulsante "Inviate" (tab delle capsule inviate) e verifica che una capsula chiamata "Past Capsule" venga visualizzata correttamente, indicando che il filtro per le capsule inviate funziona come previsto.

```
@Test
fun switchesToSentCapsulesOnSentButtonClick() {
    composeTestRule.activity.setContent {
        AllTimeCapsulesScreen(
            onNavigateToViewTimeCapsule = {_, _ -> },
            onNavigateToCreateTimeCapsule = {},
        )
    }
    val sentText = composeTestRule.activity.getString(R.string.sent)
    composeTestRule.onNodeWithText(sentText).performClick()
    composeTestRule.waitUntil(
        timeoutMillis = 5_000,
        condition = {
            composeTestRule
                .onAllNodesWithText("Past Capsule")
                .fetchSemanticsNodes().isNotEmpty()
        }
    )
    composeTestRule
        .onNodeWithText("Past Capsule")
        .assertIsDisplayed()
}
```

Test switchesToReceivedCapsulesOnReceivedButtonClick

Il test simula il clic sul pulsante "Ricevute" (tab delle capsule ricevute) e controlla che una capsula chiamata "Received Capsule" venga mostrata. Questo test verifica la corretta gestione del filtro per visualizzare le capsule ricevute.

```
@Test
fun switchesToReceivedCapsulesOnReceivedButtonClick() {
    composeTestRule.activity.setContent {
        AllTimeCapsulesScreen(
            onNavigateToViewTimeCapsule = {_, _ -> },
            onNavigateToCreateTimeCapsule = {},
        )
    }
    val receivedText = composeTestRule.activity.getString(R.string.received)
    composeTestRule.onNodeWithText(receivedText).performClick()
    composeTestRule.waitUntil(
        timeoutMillis = 5_000,
        condition = {
            composeTestRule
                .onAllNodesWithText("Received Capsule")
                .fetchSemanticsNodes().isNotEmpty()
        }
    )
    composeTestRule
        .onNodeWithText("Received Capsule")
        .assertIsDisplayed()
}
```

Test displaysCreateCapsuleFabAndRespondsToClick

Il test verifica la presenza del Floating Action Button (FAB) per la creazione di una nuova capsula temporale e ne simula il clic. Assicura che il pulsante sia visibile e risponda correttamente all'interazione dell'utente.

```
@Test
fun displaysCreateCapsuleFabAndRespondsToClick() {
    composeTestRule.activity.setContent {
        AllTimeCapsulesScreen(
            onNavigateToViewTimeCapsule = {_, _ -> },
            onNavigateToCreateTimeCapsule = {},
        )
    }
    val fabDesc = composeTestRule.activity.getString(R.string.create_time_capsule)
    composeTestRule
        .onNodeWithContentDescription(fabDesc)
        .assertIsDisplayed()
        .performClick()
}
```

Test deleteDialogAppearsWhenDeleteOpened

Il test simula l'azione di eliminazione su una capsula temporale cliccando sull'icona o pulsante "Elimina". Il test verifica che venga visualizzato un dialogo di conferma con il titolo appropriato, confermando che l'interfaccia gestisce correttamente l'intento di cancellazione.

```
@Test
fun deleteDialogAppearsWhenDeleteOpened() {
    composeTestRule.activity.setContent {
        AllTimeCapsulesScreen(
            onNavigateToViewTimeCapsule = {_, _ -> },
            onNavigateToCreateTimeCapsule = {},
        )
    }
    val deleteText = composeTestRule.activity.getString(R.string.delete)
    composeTestRule.onNodeWithContentDescription(deleteText).performClick()
    val dialogTitle = composeTestRule.activity.getString(R.string.confirm_capsule_deletion)
    composeTestRule.onNodeWithText(dialogTitle).assertIsDisplayed()
}
```

6. Implementazione in Flutter

Per lo sviluppo della nostra applicazione ibrida abbiamo usato Flutter, un framework che adotta un sistema UI completamente personalizzato e indipendente dai componenti nativi della piattaforma. In Flutter, ogni elemento dell'interfaccia – come testi, pulsanti o scroll – viene disegnato da zero tramite il proprio motore grafico.

Questa caratteristica consente una reale esperienza cross-platform, permettendo di utilizzare una singola base di codice per distribuire l'applicazione su Android, iOS, web e desktop, mantenendo coerenza visiva e comportamentale su tutti i dispositivi.

Flutter utilizza il linguaggio di programmazione Dart e adotta un approccio dichiarativo per la costruzione dell'interfaccia utente, concettualmente simile a quanto avviene con Jetpack Compose nel mondo Android con Kotlin.

La nostra applicazione in Flutter presenta le stesse funzionalità di quella Kotlin, tranne l'inserimento ed eliminazione delle immagini nel diario.

6.1 Struttura del File System del Progetto

Per implementare l'architettura MVVM anche all'interno del progetto Flutter, abbiamo scelto di utilizzare Riverpod, un moderno e potente sistema di gestione dello stato.

Riverpod ci ha permesso di separare in modo chiaro le responsabilità tra Model, View e ViewModel. In particolare, i ViewModel vengono esposti come provider, e le view (i widget) possono reagire automaticamente ai cambiamenti di stato, mantenendo un approccio dichiarativo e reattivo, in linea con la filosofia di Flutter.

Questa scelta ha reso possibile un'organizzazione del progetto coerente con quella già adottata nell'applicazione Android con Kotlin, facilitando la gestione del business logic e migliorando la leggibilità del codice.

Struttura delle cartelle principali

Nella versione Flutter dell'applicazione, la struttura delle cartelle rimane quasi identica a quella adottata nel progetto Android sviluppato in Kotlin con alcune modifiche chiave dovute alle differenze tra le due piattaforme.

- La cartella *viewmodel* viene rinominata in *notifier*, in quanto su Flutter abbiamo utilizzato Riverpod per adottare l'architettura MVVM. I Notifier fungono da equivalenti dei ViewModel, gestendo lo stato, rispondendo agli eventi dell'interfaccia utente e coordinando il recupero dei dati tramite i repository.

- La cartella *navigation* non è presente. A differenza di Jetpack Compose, dove abbiamo centralizzato la navigazione in un modulo apposito, in Flutter abbiamo definito le routes direttamente all'interno di ciascuno screen.

File principali a livello di root

- *main.dart*: è il punto di ingresso dell'app Flutter. Si occupa dell'inizializzazione dei servizi, dell'avvio dell'interfaccia utente e della configurazione generale dell'app, come tema, localizzazione e struttura di navigazione.

- *router.dart*: contiene la definizione delle rotte principali dell'app gestite tramite *go_router*. Qui viene configurato il sistema di navigazione, specificando i percorsi, gli screen associati e gli eventuali parametri.

- *storage_service.dart*: come in kotlin, gestisce l'interazione con il livello di persistenza remota (Firebase). Fornisce metodi per eseguire operazioni asincrone come lettura, scrittura, aggiornamento e cancellazione dei dati. Questo servizio è utilizzato dai repository per accedere in modo centralizzato ai dati remoti.

File di configurazione

- *pubspec.yaml*: è il file di configurazione principale di un progetto Flutter. In questo file vengono definiti il nome, la descrizione e la versione dell'applicazione. Vengono inoltre specificate tutte le dipendenze esterne necessarie al progetto, come librerie e plugin. Il file include anche l'elenco degli asset, come immagini e font, che devono essere inclusi nell'app.

6.2 Approfondimenti sul codice

Per quanto riguarda lo sviluppo in Flutter, come accennato prima, abbiamo scelto un approccio del tutto equivalente a quello utilizzato in Android, mantenendo quindi la stessa struttura di progetto. Gli unici dettagli degni di nota sono l'utilizzo di `Riverpod`, `Freezed` e `go_router`, tre librerie molto popolari per la programmazione in Flutter utili per velocizzare lo sviluppo di un'app.

Riguardo a `Riverpod`, si tratta di una libreria per la gestione dello stato reattivo che consente, in maniera del tutto analoga allo StateFlow di Jetpack Compose, di avere uno stato osservabile della UI senza necessità di aggiornamenti manuali da parte del ViewModel (ora chiamato Notifier). Il notifier mette quindi a disposizione lo stato e dei metodi per l'aggiornamento dello stesso. Per poter osservare lo stato del notifier, il widget va ad utilizzare un apposito provider che, dato il tipo di notifier e il corrispettivo tipo di stato, va a costruire il notifier iniettando le dipendenze necessarie. In particolare, durante la build del *ConsumerWidget*, si va ad osservare lo stato fornito dal provider tramite *ref.watch* e si vanno a recuperare i metodi del notifier tramite *ref.read*. Al cambiamento dello stato avverrà una ricomposizione della UI automatica.

Per quanto riguarda `Freezed`, è una libreria in grado di generare codice per estendere le funzionalità di una classe. E' necessario dichiarare un factory con gli attributi della classe e `Freezed` provvederà ad aggiungere metodi come *copyWith* e *toString*. Abbiamo utilizzato `Freezed` per tutti i model e tutte le classi degli stati dei Notifier.

Infine `go_router` è una libreria per gestire la navigazione in Flutter in maniera dichiarativa. Permette la dichiarazione di uno *ShellRoute*, ovvero una route di base che contiene gli elementi comuni a tutta l'interfaccia dell'app (Scaffold, AppBar e BottomNavigationBar), e di diverse rotte *GoRoute*, elementi equivalenti ai *composable* del *NavHost* di *Jetpack Compose*.

6.3 Descrizione unit tests

Come in kotlin, anche gli unit test in Flutter vanno a garantire il corretto funzionamento di alcune funzionalità della nostra applicazione. Sono state testate le funzionalità principali del sistema quali: la visualizzazione dei diari, la visualizzazione delle capsule del tempo, la creazione di un diario e la creazione di una capsula del tempo. Grazie a questi test è stato possibile verificare che l'applicazione si comporti come previsto oltre ad individuare eventuali bug in fase di sviluppo.

In tutti i test viene predisposto un setup iniziale volto a isolare la logica della schermata dai servizi esterni. In particolare, si ricorre a delle classi mock per simulare il comportamento delle dipendenze principali: *DiaryRepository*, *TimeCapsuleRepository*, *FirebaseAuth* e l'utente Firebase.

All'interno del blocco *setUp*, che viene eseguito prima di ogni test, vengono create le istanze fittizie di repository e autenticazione. Viene poi configurato il mock dell'utente, forzando la risposta di *mockAuth.currentUser* con un oggetto *MockUser* a cui viene assegnato un id predefinito. Questo approccio consente di simulare uno scenario in cui un utente è autenticato e di testare il comportamento della logica collegata al suo profilo, senza dipendere da Firebase o da una rete attiva.

6.3.1 all_diaries_screen_test.dart

Il test verifica che la schermata *AllDiariesScreen* mostri correttamente il titolo, la descrizione e le icone di modifica e cancellazione di un diario quando lo stato è già precompilato. Serve a garantire che la UI visualizzi i dati previsti in condizioni normali

```
class MockRepo extends Mock implements DiaryRepository {}
class MockAuth extends Mock implements FirebaseAuth {}
class MockLoc extends Mock implements AppLocalizations {}

void main() {
  late MockDiaryRepository mockRepo;
  late MockFirebaseAuth mockAuth;
  late MockUser mockUser;
  final userId = 'test-user-id';

  setUp(() {
    mockRepo = MockDiaryRepository();
    mockAuth = MockFirebaseAuth();
    mockUser = MockUser();

    when(mockAuth.currentUser).thenReturn(mockUser);
    when(mockUser.uid).thenReturn(userId);
  });
```

```
  testWidgets('Displays diary title and description', (tester) async {
    final fakeDiary = Diary(
      id: '1', title: title, description: description,
      coverId: 'cover1', pageIds: ['p1', 'p2'], creationDate: Timestamp(1, 1),
    );
    final fakeCover = DiaryCover(url: 'https://example.com/cover.jpg');
    final fakeState = AsyncValue.data(
      AllDiariesState(
        diaries: [fakeDiary], diaryCoversMap: {'cover1': fakeCover},
        deleteDialogState: false,
        currentIndex: 0, pageController: PageController(),
      ),
    );
    final notifier = AllDiariesNotifier(
      profileId: userId,
      repository: mockRepo,
    );
    notifier.state = fakeState;
    await tester.pumpWidget(
      ProviderScope(
        overrides: [
          allDiariesNotifierProvider(userId).overrideWith((_) => notifier),
        ],
        child: MaterialApp(
          localizationsDelegates: const [
            AppLocalizations.delegate,
            DefaultWidgetsLocalizations.delegate,
            DefaultMaterialLocalizations.delegate,
          ],
          supportedLocales: const [Locale('en')],
          home: AllDiariesScreen(
            profileId: userId,
            onNavigateToDiary: (_) {},
            onNavigateToEditDiary: (_) async => fakeDiary,
            onNavigateToCreateDiary: () async => fakeDiary,
          ),
        ),
      ),
    );
    await tester.pumpAndSettle();
    expect(find.text('Titolo Test'), findsOneWidget);
    expect(find.text('Descrizione Test'), findsOneWidget);
    expect(find.byIcon(Icons.edit), findsOneWidget);
    expect(find.byIcon(Icons.delete_outline), findsOneWidget);
  });
}
```

6.3.2 all_time_capsule_screen_test.dart

Questo test ha lo scopo di verificare che la schermata *AllTimeCapsulesScreen* mostri correttamente il titolo di una capsula del tempo e che il pulsante per crearne una nuova (*FloatingActionButton*) sia presente. Per farlo, viene preparato uno stato fittizio (*fakeState*) contenente una capsula inviata dall'utente, che viene poi fornito al notifier sovrascrivendo il provider. Infine, l'interfaccia viene renderizzata e si controlla che il titolo della capsula sia visibile e che il pulsante per l'aggiunta sia correttamente presente nella UI.

```
class MockRepo extends Mock implements TimeCapsuleRepository {}
class MockAuth extends Mock implements FirebaseAuth {}
class MockLoc extends Mock implements AppLocalizations {}

void main() {
  late MockTimeCapsuleRepository mockRepo;
  late MockFirebaseAuth mockAuth;
  late MockUser mockUser;
  final userId = 'test-user-id';

  setUp(() {
    mockRepo = MockTimeCapsuleRepository();
    mockAuth = MockFirebaseAuth();
    mockUser = MockUser();

    when(mockAuth.currentUser).thenReturn(mockUser);
    when(mockUser.uid).thenReturn(userId);
  });
```

```
  testWidgets('Displays time capsule title', (tester) async {
    final message = 'Messaggio per il futuro';
    final fakeTimeCapsule = TimeCapsule(
      id: '1', userId: userId,
      title: message, content: 'Ciao futuro me! Spero che tu abbia raggiunto i tuoi sogni.',
      deadline: Timestamp.fromDate(DateTime(2030, 1, 1)),
      emails: [], phones: [], receiversIds: [userId],
      creationDate: Timestamp.fromDate(DateTime(2029, 1, 1)),
      isSent: false,
    );
    final fakeState = AsyncValue.data(
      AllTimeCapsulesState(
        sentTimeCapsules: {'1': fakeTimeCapsule},
        receivedTimeCapsules: {},
        buttonState: TimeCapsuleType.scheduled,
      ),
    );
    final notifier = AllTimeCapsulesNotifier(
      profileId: userId, repository: mockRepo, auth: mockAuth,
    );
    notifier.state = fakeState;
    await tester.pumpWidget(
      ProviderScope(
        overrides: [
          allTimeCapsulesNotifierProvider(userId).overrideWith((_) => notifier),
        ],
        child: MaterialApp(
          localizationsDelegates: const [
            AppLocalizations.delegate,
            DefaultWidgetsLocalizations.delegate,
            DefaultMaterialLocalizations.delegate,
          ],
          supportedLocales: const [Locale('en')],
          home: AllTimeCapsulesScreen(
            profileId: userId,
            onNavigateToCreateTimeCapsule: () async => fakeTimeCapsule,
            onNavigateToViewTimeCapsule: (_, _) {},
          ),
        ),
      ),
    );
    await tester.pumpAndSettle();
    expect(find.text(message), findsOneWidget);
    expect(find.byType(FloatingActionButton), findsOneWidget);
  });
}
```

6.3.3 edit_diary_screen_test.dart

In questo test viene dichiarato un *FakeEditDiaryNotifier*, una versione fittizia del notifier reale, che sovrascrive il metodo *onSaveDiary* per tenere traccia dell'avvenuta chiamata tramite una semplice variabile booleana (*saveCalled*). Questo permette di verificare nei test che il salvataggio del diario sia stato effettivamente richiesto, senza eseguire la logica reale o accedere a servizi esterni.

Il test verifica che la schermata di modifica di un diario venga renderizzata correttamente e che il pulsante "*Save*" attivi la logica di salvataggio. Utilizzando uno stato e un notifier fittizi, controlla la presenza dei widget principali e simula il tap sul bottone, assicurandosi che la funzione di salvataggio venga effettivamente chiamata.

```dart
class MockRepo extends Mock implements DiaryRepository {}
class MockAuth extends Mock implements FirebaseAuth {}
class MockLoc extends Mock implements AppLocalizations {}

class FakeEditDiaryNotifier extends EditDiaryNotifier {
  bool saveCalled = false;
  FakeEditDiaryNotifier({
    required super.diaryId, required super.repository,
    required super.auth, required super.localizations,
  });
  @override
  Future<void> onSaveDiary(void Function(Diary) onComplete) async {
    saveCalled = true;
  }
}

void main() {
  late MockDiaryRepository mockRepo;
  late MockFirebaseAuth mockAuth;
  late MockUser mockUser;
  late MockAppLocalizations mockLoc;

  setUp(() {
    mockRepo = MockDiaryRepository();
    mockAuth = MockFirebaseAuth();
    mockUser = MockUser();
    mockLoc = MockAppLocalizations();

    when(mockAuth.currentUser).thenReturn(mockUser);
    when(mockUser.uid).thenReturn('test-user-id');
    when(mockRepo.getAllDiaryCovers()).thenAnswer((_) async => [
      DiaryCover(id: '1', url: 'https://example.com/cover1.jpg'),
      DiaryCover(id: '2', url: 'https://example.com/cover2.jpg'),
    ]);
  });
```

```
  testWidgets('Renders and triggers save button to save Diary', (tester) async {
    final fakeDiary = Diary(
      id: 'test-diary-id', uid: 'test-user-id',
      title: 'Titolo Test', description: 'Descrizione Test',
      coverId: '1', pageIds: ['p1', 'p2'], creationDate: Timestamp(1, 1),
    );
    final fakeState = AsyncValue.data(
        EditDiaryState(
          diary: fakeDiary,
          allCoversList: [
            DiaryCover(id: '1', url: 'https://example.com/cover1.jpg'),
            DiaryCover(id: '2', url: 'https://example.com/cover2.jpg'),
          ],
          titleError: '', descriptionError: '',
        )
    );
    final fakeNotifier = FakeEditDiaryNotifier(
      diaryId: 'test-diary-id', repository: mockRepo, auth: mockAuth, localizations: mockLoc,
    )..state = fakeState;
    await tester.pumpWidget(
      ProviderScope(
        overrides: [
          editDiaryNotifierProvider('test-diary-id').overrideWith((ref) => fakeNotifier),
        ],
        child: MaterialApp(
          localizationsDelegates: const [
            AppLocalizations.delegate,
            DefaultWidgetsLocalizations.delegate, DefaultMaterialLocalizations.delegate,
          ],
          supportedLocales: const [Locale('en')],
          home: Scaffold(
            body: EditDiaryScreen(diaryId: 'test-diary-id', onComplete: (_) {}),
          ),
        ),
      ),
    );
    await tester.pumpAndSettle();
    expect(find.text('Create diary'), findsOneWidget);
    expect(find.byType(SingleLineFieldWithError), findsOneWidget);
    expect(find.byType(ContentTextFieldWithError), findsOneWidget);
    expect(find.byType(GridView), findsOneWidget);
    expect(find.byType(DiaryCoverPreview), findsNWidgets(2));
    await tester.tap(find.text('Save'));
    await tester.pump();
    expect(fakeNotifier.saveCalled, isTrue);
  });
}
```

6.3.4 create_time_capsule_screen_test.dart

In questo test viene dichiarato un *FakeCreateTimeCapsuleNotifier*, una versione fittizia del notifier reale che sovrascrive il metodo *createTimeCapsule*, utilizzando una variabile booleana (*createCalled*) per tracciare se la funzione è stata effettivamente invocata. Questo consente di testare il comportamento della schermata senza attivare la logica reale o dipendere da servizi esterni.

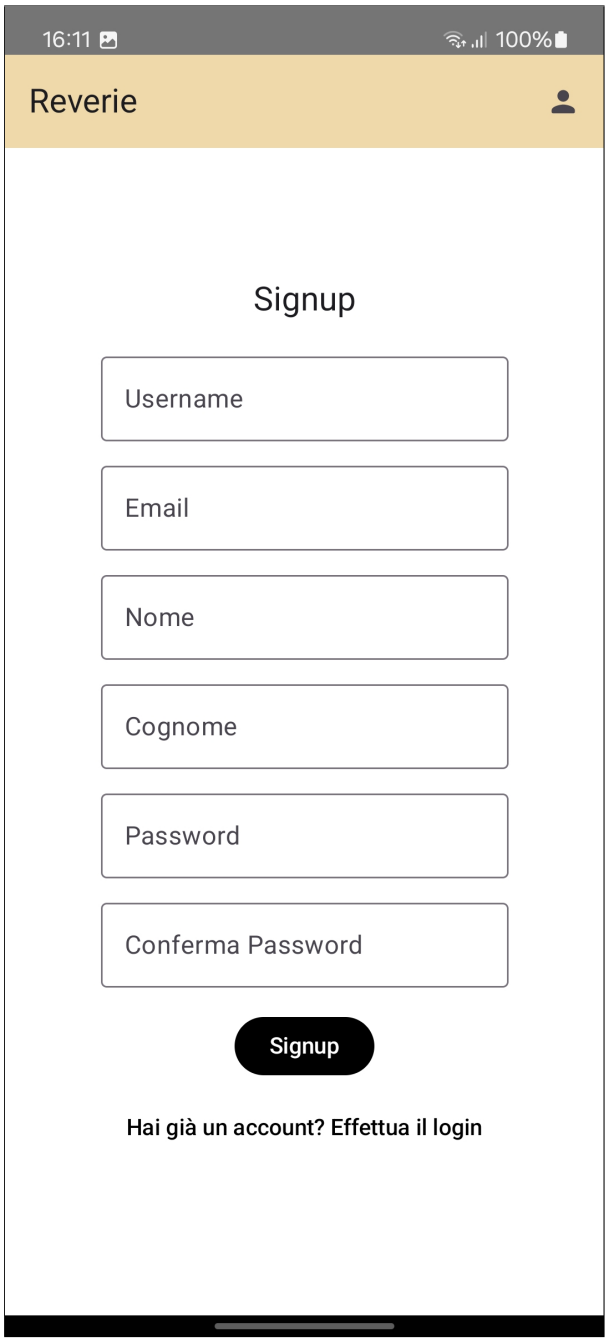
Il test verifica che la schermata di creazione di una capsula del tempo venga renderizzata correttamente, controllando la presenza dei widget principali come campi di input e pulsanti. Viene poi simulato un tap sul bottone "*Create*", e si verifica che la funzione di creazione della capsula venga effettivamente chiamata.

```
class MockTimeCapsuleRepo extends Mock implements TimeCapsuleRepository {}
class MockUserRepo extends Mock implements UserRepository {}
class MockAuth extends Mock implements FirebaseAuth {}
class MockLoc extends Mock implements AppLocalizations {}
class FakeCreateTimeCapsuleNotifier extends CreateTimeCapsuleNotifier {
  bool createCalled = false;

  FakeCreateTimeCapsuleNotifier({
    required super.repository,
    required super.userRepository,
    required super.auth,
    required super.localizations,
  });
  @override
  void createTimeCapsule(void Function(TimeCapsule) onComplete) {
    createCalled = true;
  }
}

void main() {
  late MockTimeCapsuleRepository mockTimeCapsuleRepo;
  late MockUserRepository mockUserRepo;
  late MockFirebaseAuth mockAuth;
  late MockUser mockUser;
  late MockAppLocalizations mockLoc;
  setUp(() {
    mockTimeCapsuleRepo = MockTimeCapsuleRepository();
    mockUserRepo = MockUserRepository();
    mockUser = MockUser();
    mockAuth = MockFirebaseAuth();
    mockLoc = MockAppLocalizations();
    when(mockAuth.currentUser).thenReturn(mockUser);
    when(mockUser.uid).thenReturn('test-user-id');
  });
```

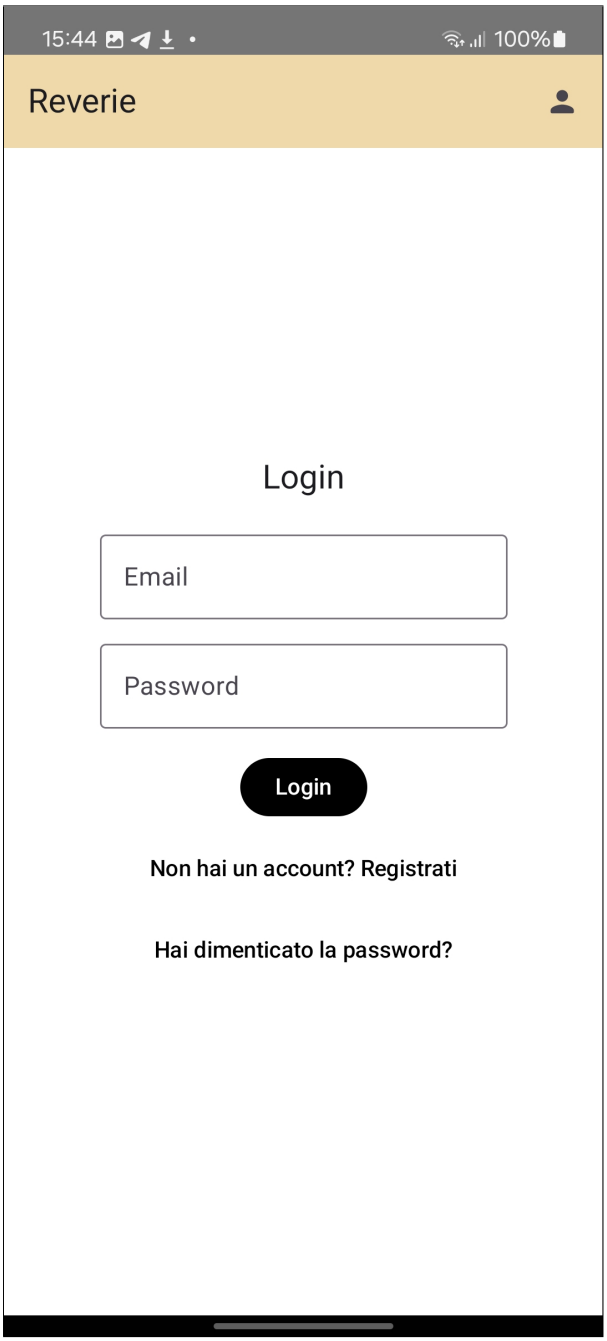
```
  testWidgets('Renders and triggers create button to create TimeCapsule', (tester) async {
    final create = 'Create';
    final selectDate = 'Select Date';
    final fakeTimeCapsule = TimeCapsule(
      id: '1', userId: 'test-user-id',
      title: '', content: '',
      deadline: Timestamp.now(),
      emails: [], phones: [], receiversIds: [],
      creationDate: Timestamp.now(),
      isSent: false,
    );
    final fakeState = CreateTimeCapsuleState(timeCapsule: fakeTimeCapsule);
    final fakeNotifier = FakeCreateTimeCapsuleNotifier(
      repository: mockTimeCapsuleRepo,
      userRepository: mockUserRepo,
      auth: mockAuth,
      localizations: mockLoc,
    )..state = fakeState;
    await tester.pumpWidget(
      ProviderScope(
        overrides: [
          createTimeCapsuleNotifierProvider.overrideWith((_) => fakeNotifier),
        ],
        child: MaterialApp(
          localizationsDelegates: const [
            AppLocalizations.delegate,
            DefaultWidgetsLocalizations.delegate,
            DefaultMaterialLocalizations.delegate,
          ],
          supportedLocales: const [Locale('en')],
          home: Scaffold(
            body: CreateTimeCapsuleScreen(onComplete: (_) {}),
          ),
        ),
      ),
    );
    await tester.pump();
    expect(find.text(create), findsOneWidget);
    expect(find.text(selectDate), findsOneWidget);
    expect(find.byType(SingleLineFieldWithError), findsNWidgets(2));
    expect(find.byType(ContentTextFieldWithError), findsOneWidget);
    expect(find.byType(PhoneNumberField), findsOneWidget);
    await tester.ensureVisible(find.text(create));
    await tester.tap(find.text(create));
    await tester.pump();
    expect(fakeNotifier.createCalled, isTrue);
  });
}
```

7. Conclusione

Per concludere, mostriamo delle foto dell'app ultimata.



Pagina di Signup

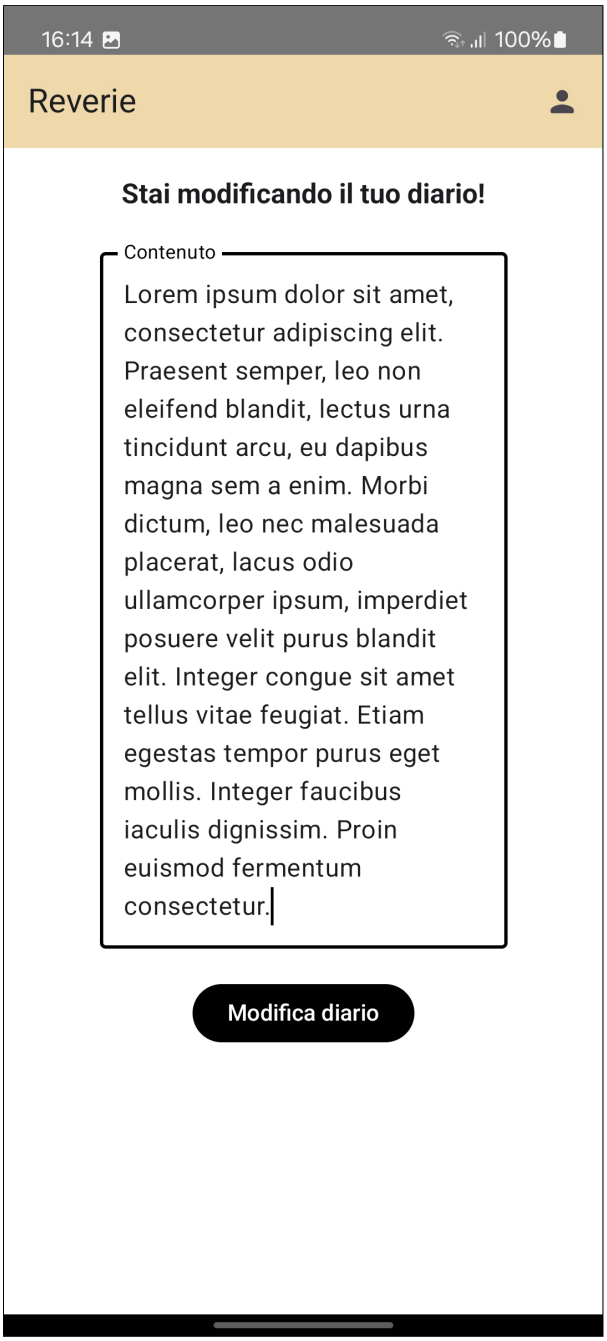


Pagina di Login

Pagina dei Diari

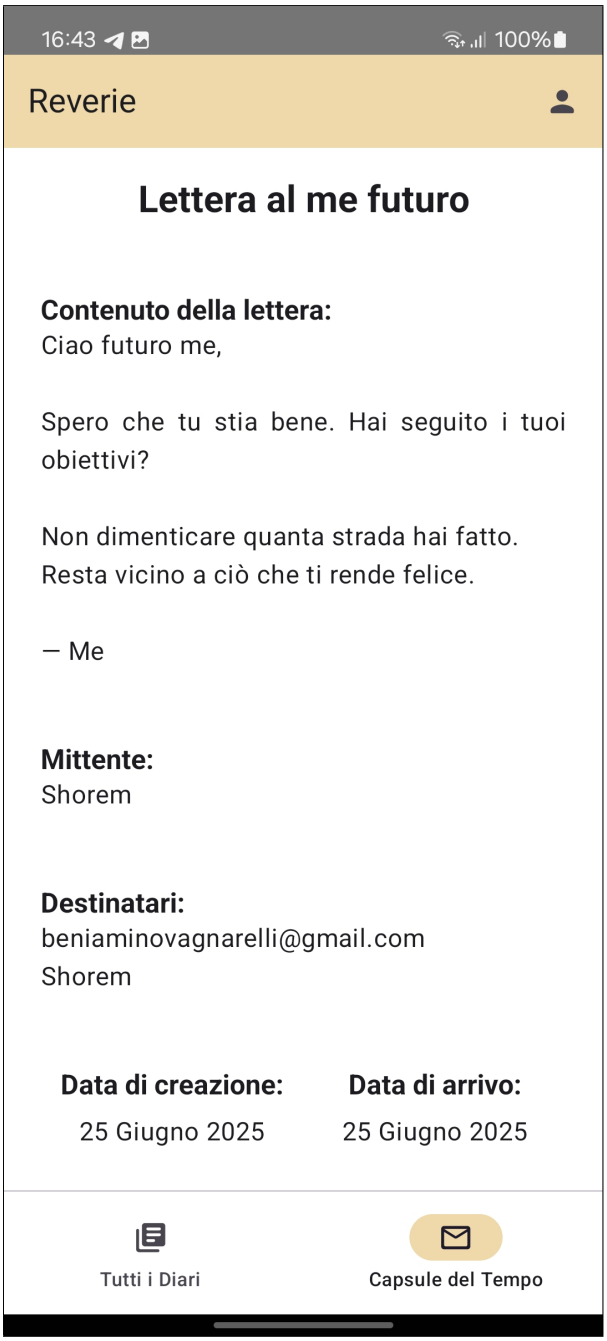
Pagina di Personalizzazione Diario

Pagina di Visualizzazione Diario



Pagina di Modifica Contenuto Diario

Pagina delle Capsule del Tempo



Pagina di Visualizzazione Capsula del Tempo